



Republic of Tunisia
Ministry of Higher Education
and Scientific Research

University of Carthage
Higher Institute of Information
and Communication Technologies



END-OF-YEAR PROJECT REPORT

Field of Study: Computer Engineering
Specialty: Networks and Systems Engineering

The Network Sentinel

By
Mariem Rmili

Produced within

NO BREACH

Academic Year : 2026–2027

Acknowledgements

I would like to express my gratitude to my academic supervisor, **Mr. Mahmoud Mansouri**, for his guidance, support, and valuable feedback throughout this project. His availability and encouragement made a real difference in completing this work successfully.

I would also like to thank **Mrs. Nouha Ben Brahim**, founder of **No Breach Training Hub**, for creating this internship program and providing me with the environment and resources to work on a real, hands-on cybersecurity project.

Abstract

This report presents a **Final Year Project** carried out as part of the **No Breach Training Hub** cybersecurity internship program.

The goal was to design, attack, and defend a complete virtual network using real security tools and techniques.

1. The first part covered building a segmented network with a firewall, strict access rules, and an encrypted **VPN** for secure remote access.
2. The second part focused on simulating real attacks using **Kali Linux**, analyzing them in **Wireshark**, and deploying Suricata to automatically detect and block them in real-time. Alerts were forwarded to a central machine using Syslog, simulating a basic **SIEM** pipeline.

The final result is a fully working security system built entirely with free, open-source tools.

Keywords: Network Security, OPNsense, Suricata, IDS/IPS, VPN, Wireshark, SIEM, Kali Linux, Virtualization.

Contents

Acknowledgements	i
Abstract	ii
List Of Abbreviations	vi
List of Figures	ix
General Introduction	1
1 Presentation Of The Project Context	2
1 Introduction	2
2 Presentation Of the Company	2
2.1 Activity of No Breach Tunisia	2
2.2 Mission	2
2.3 Core Activities	2
2.4 Client Engagements	3
2.5 Training & Community Building	3
3 Conclusion	3
2 Environment Setup & Tools	4
1 Virtualization Platform: VirtualBox	4
2 Firewall: OPNsense	5
3 Attacker Machine: Kali Linux	5
4 Target Machine: Ubuntu Server	6
5 Network Diagram Tool: Draw.io	6
3 Phase I: Architecture and Secure Connectivity	7
1 Sprint 1: Network Segmentation and Firewall Configuration	7
1.1 Virtual Network Design	7
1.2 Firewall Installation	7
1.2.1 OPNsense Installation	8
1.2.2 Interface Assignment (WAN, LAN, and DMZ)	8
1.2.3 IP Address Configuration (WAN, LAN, and DMZ)	9
1.3 Access Control Lists (ACLs)	15
1.4 NAT and Port Forwarding	16
1.4.1 Ubuntu Server Installation & Configuring	17
1.4.2 Launching the Python Web Server	19
1.4.3 Creating a Port Forwarding Rule	19

1.5	Sprint 1 Validation and Testing	19
1.5.1	Test 1: Web Server Accessibility from Kali Linux (WAN Side)	19
1.5.2	Test 2: DMZ-to-LAN Isolation Verification	21
1.6	Bonus: Python-Based Reachability Scanner	21
1.7	Conclusion	22
2	Sprint 2: Encrypted VPN Tunneling	23
2.1	WireGuard Overview & Availability	23
2.2	Cryptographic Key Generation	24
2.2.1	OPNsense Key Generation	24
2.2.2	Kali Key Generation	25
2.3	Tunnel Configuration	25
2.3.1	Tunnel Subnet	25
2.3.2	WAN Firewall Rule : Allow WireGuard UDP 51820	26
2.4	Client Provisioning & Verification	26
2.4.1	Kali Peer Registration on OPNsense	26
2.4.2	Enabling WireGuard on OPNsense	26
2.4.3	Kali Client Configuration	27
2.4.4	Connecting Kali to the VPN	28
2.5	Verify the Connection	28
2.6	Sprint 2 Validation and Testing	30
2.7	Bonus : VPN Monitor Script	32
2.8	Conclusion	34
4	PHASE II : Traffic Analysis & Active Defense	35
1	Sprint 3 : Deep Packet Inspection (DPI) & PCAP Analysis	35
1.1	Attack Simulation Overview	35
1.1.1	Nmap Port Scan	36
1.1.2	hping3 : SYN Flood	37
1.1.3	Hydra : Brute Force	38
1.2	Packet Capture	39
1.2.1	Start Capture on OPNsense WAN Interface	40
1.2.2	Download the .pcap File	40
1.3	Wireshark Analysis	41
1.3.1	Nmap Scan Analysis	41
1.3.2	SYN Flood Analysis	42
1.3.3	Hydra Brute Force Analysis	43
1.4	Bonus : Programmatic PCAP Parsing	44
1.4.1	Installing pyshark	44
1.4.2	Python Script : Top 5 IP Addresses	44
1.5	Conclusion	45
2	Sprint 4 : Intrusion Detection System (IDS) & Prevention	46
2.1	Suricata Deployment	46
2.1.1	Install Suricata on OPNsense	46
2.1.2	Configure Suricata to Monitor the WAN Interface	47
2.1.3	Add the DMZ Interface	47
2.1.4	Download the Emerging Threats Community Ruleset	48
2.2	Custom Rule Creation	49

2.2.1	Custom Rule: Nmap Port Scan Detection	49
2.2.2	Custom Rule: Hydra Brute-Force Detection	49
2.2.3	Testing: Confirming Alerts Fire Before Switching to IPS .	50
2.3	Transition to IPS Mode	51
2.3.1	Switch Suricata to Inline IPS Mode	51
2.3.2	Change Rules from Alert to Drop	52
2.3.3	Verify No False Positives	53
2.4	Final Test: All Three Attacks Blocked	54
2.4.1	Nmap Scan	54
2.4.2	hping3 SYN Flood	55
2.4.3	Hydra Brute-Force	55
2.4.4	Suricata Alerts Confirm Full Blocking	55
2.5	Bonus: Syslog Telemetry Forwarding	56
2.5.1	Enable Suricata Syslog Output	56
2.5.2	Configure OPNsense Syslog Forwarder	56
2.5.3	Write a Python UDP Listener on the Windows Host . . .	57
2.5.4	Confirm Alerts Appear in Real-Time	57
2.6	Conclusion	58
General Conclusion		60

List Of Abbreviations

ACL *Access Control List*

CPU *Central Processing Unit*

DMZ *Demilitarized Zone*

DNS *Domain Name System*

DPI *Deep Packet Inspection*

GUI *Graphical User Interface*

HTTP *Hypertext Transfer Protocol*

HTTPS *Hypertext Transfer Protocol Secure*

IDS *Intrusion Detection System*

IP *Internet Protocol*

IPS *Intrusion Prevention System*

LAN *Local Area Network*

NAT *Network Address Translation*

PCAP *Packet Capture*

SSH *Secure Shell*

TCP *Transmission Control Protocol*

UDP *User Datagram Protocol*

VPN *Virtual Private Network*

WAN *Wide Area Network*

Wi-Fi *Wireless Fidelity*

List of Figures

2.1	VirtualBox Logo	4
2.2	OPNsense Logo	5
2.3	Kali Linux Logo	5
2.4	Ubuntu Server Logo	6
2.5	Draw.io Logo	6
3.1	OPNsense VM in VirtualBox	8
3.2	OPNsense VM Configuration : VirtualBox Details	8
3.3	Network Adapter Configuration	9
3.4	VirtualBox Network Manager : Host-Only Adapter	9
3.5	OPNsense Console: LAN IP Configuration	9
3.6	Avoiding IP Conflict	10
3.7	Host PC: Network Connection Details	10
3.8	Host PC: Wi-Fi Adapter Configuration (ipconfig)	11
3.9	OPNsense Console: WAN Interface Verification (ifconfig em1)	11
3.10	VirtualBox Network Manager: DMZ Host-Only Adapter 2	12
3.11	OPNsense Console: DMZ Interface Assignment (em2)	12
3.12	Host PC : VirtualBox Host-Only Adapter 2 configuration (10.0.2.2)	13
3.13	OPNsense Interface Assignment Summary	13
3.14	Lab Network Topology	14
3.15	LAN Connectivity Test (ping 192.168.2.1)	14
3.16	OPNsense Web GUI Login Page	15
3.17	OPNsense Dashboard : Initial Configuration Screen	15
3.18	Firewall Rules: LAN Interface	16
3.19	Firewall Rules: DMZ Interface (OPT1)	16
3.20	Ubuntu Server VM in VirtualBox	17
3.21	Ubuntu Server Network Adapter Configuration	17
3.22	Ubuntu Server Static IP Configuration	18
3.23	Profile Configuration	18
3.24	Ubuntu Server IP Verification (ip a)	18
3.25	Python HTTP server running on Ubuntu in the DMZ	19
3.26	OPNsense NAT : Destination NAT Rule	19
3.27	WAN Firewall Rule: Allow HTTP to DMZ	20
3.28	Kali Linux IP Configuration (ip a)	20
3.29	HTTP Request from Kali to Ubuntu Web Server (curl)	20
3.30	Ubuntu Server HTTP Request Log	21
3.31	Ubuntu DMZ Server : Ping to LAN (192.168.2.10)	21
3.32	Python Reachability Scanner Output	22

3.33	OPNsense VPN Menu: WireGuard Available	24
3.34	OPNsense Key Generation:Auto-generated WireGuard Key Pair	24
3.35	Kali Key Generation:WireGuard Key Pair Generated on Kali	25
3.36	WireGuard Server Instance on OPNsense	25
3.37	WAN Firewall Rule: Allow WireGuard VPN	26
3.38	WireGuard Peer: Kali Client Registered on OPNsense	26
3.39	WireGuard Service Enabled on OPNsense	27
3.40	WireGuard Configuration File on Kali (wg0.conf)	27
3.41	Setting File Permissions on wg0.conf	27
3.42	Connecting Kali to the VPN	28
3.43	WireGuard Connection Status on Kali (wg show)	28
3.44	WireGuard Status on OPNsense	29
3.45	WireGuard Interface Assigned as VPN (OPT2)	29
3.46	Dedicated VPN Interface Tab in Firewall Rules	29
3.47	Firewall Rule: Allow VPN to LAN	30
3.48	Ping to OPNsense LAN Interface over VPN	30
3.49	OPNsense Web GUI Accessed from Kali over VPN	31
3.50	OPNsense GUI Source Code Retrieved via curl	31
3.51	OPNsense Console: All Interfaces Active	31
3.52	VPN Disconnected : Ping to LAN Fails	32
3.53	Kali WireGuard Configuration with Modified Private Key	33
3.54	VPN Monitor Script: Red Alert :No Handshake Detected	33
3.55	VPN Monitor Script: Green Status : Handshake Restored	34
4.1	Nmap Scan Result: Reconnaissance Successful	36
4.2	Ubuntu Server Log: Nmap Scan Detected	37
4.3	SYN Flood Attack Completed: 14 Million Packets Sent	37
4.4	Ubuntu Server : CPU Soft Lockup During SYN Flood	38
4.5	Hydra Brute-Force Attack Running on Kali	38
4.6	Ubuntu Server: Brute-Force Traffic Received	39
4.7	Start Capture on OPNsense WAN Interface	40
4.8	Packet Capture Job Running	40
4.9	Downloading the .pcap File from OPNsense	40
4.10	attacks.pcap File	41
4.11	Wireshark Overview: attacks.pcap	41
4.12	Nmap Port Sweep Captured in Wireshark	42
4.13	SYN Flood: Thousands of SYN Packets Captured	42
4.14	SYN Flood: Incomplete Handshake Signature	42
4.15	Hydra Brute Force: HTTP Auth Attempts Captured	43
4.16	Hydra Brute Force: Complete Connection Per Attempt	43
4.17	pyshark Installation on Kali	44
4.18	pyshark Script Output: Top 5 IP Addresses	44
4.19	Suricata logo	46
4.20	Suricata package installed in OPNsense Firmware	47
4.21	Suricata configured on WAN interface	47
4.22	Suricata configured on OPT1 (DMZ) + WAN	48
4.23	ET open/emerging-scan	48
4.24	ET open/emerging-dos	48

4.25	ET open/emerging-web _{server}	49
4.26	Custom rule: Nmap Port Scan Detected	49
4.27	Custom rule: Hydra Brute-Force Detected	49
4.28	Nmap scan result in IDS mode	50
4.29	Hydra stopped by Suricata	50
4.30	Suricata Alerts tab: both rules firing	50
4.31	Suricata switched to Netmap (IPS) mode	51
4.32	Hardware offloading disabled	52
4.33	Custom rules set to Drop	52
4.34	curl from Kali returns the web server page	53
4.35	Suricata Alerts tab: no false positives	53
4.36	Web server accessible from Windows browser	54
4.37	Nmap scan: all ports filtered	54
4.38	hping3: 100% packet loss, 0 replies received	55
4.39	Hydra: 0 valid passwords	55
4.40	Suricata Alerts	55
4.41	Suricata Syslog output enabled	56
4.42	OPNsense remote Syslog forwarder configured	57
4.43	Python listener running and receiving logs	57
4.44	syslog-ng Suricata forward config	58
4.45	curl with malicious User-Agent sent from Kali	58
4.46	Suricata alerts appearing on Windows Python listener	58

General Introduction

This internship was carried out within the **No Breach Training Hub** cybersecurity program, as part of my training in network and systems engineering. Its main objective was to move beyond theoretical knowledge and build a complete network security infrastructure from scratch, in a virtualized environment.

Cyber attacks are a daily reality : networks get scanned, servers get flooded, and credentials get stolen, often without anyone noticing until the damage is already done. **Most networks have no system in place to detect and stop these threats automatically.** This project addresses that problem directly.

By building a complete virtual lab, simulating real attacks against it, and deploying automated defenses, we prove that with the right open-source tools, **a network can detect and block real attacks the moment they happen , without any human intervention.**

The work was structured around two main phases: first, **designing and securing the network infrastructure**, then **simulating real attacks, analyzing them, and deploying automated defenses** to stop them.

This report details the complete process : from **building the infrastructure**, to **breaking it**, to **defending it** , following the same workflow used by real security engineers every day.

Chapter 1

Presentation Of The Project Context

1 Introduction

This chapter presents the general context of the project. It begins with an introduction to the host organization, No Breach Training Hub, covering its mission, core activities, client engagements, and its role in training and community building within the cybersecurity field in Tunisia. It then presents the problem this project addresses and the solution we built to solve it.

2 Presentation Of the Company

2.1 Activity of No Breach Tunisia

No Breach is a Tunisian startup founded in 2023 by Nouha Ben Brahim, specializing in offensive cybersecurity. Since its creation, the company has positioned itself as a key player in Tunisia's cybersecurity landscape, combining technical expertise with a strong commitment to knowledge-sharing and community building.

2.2 Mission

No Breach's mission is to simplify cybersecurity and make it more accessible, helping businesses secure their applications and products with confidence — while empowering the next generation of security professionals.

2.3 Core Activities

No Breach's expertise spans several areas of offensive security:

- **Penetration Testing** : Simulating real-world attacks on web applications, networks, and systems to uncover vulnerabilities before malicious actors can exploit them.
- **Blockchain Security** : Auditing and assessing the security of blockchain-based applications and smart contracts, an increasingly critical area as decentralized technologies grow in adoption.

- **Security Consulting** : Advising organizations on strengthening their overall security posture and remediating identified risks.

2.4 Client Engagements

Over the years, No Breach has partnered with a **diverse range of companies** across multiple sectors to test and secure their **web applications** and **digital infrastructure**. Due to the confidential nature of penetration testing engagements, client identities remain undisclosed , a standard practice in the industry that reflects the **high level of trust** placed in No Breach's work.

2.5 Training & Community Building

Beyond its consulting activities, No Breach is deeply invested in **cybersecurity education**. The startup has built a structured, **hands-on learning environment** for students, offering training programs that go beyond theory to include real, practical projects , covering key topics such as:

- **Network reconnaissance** and **vulnerability scanning**,
- **MITRE ATT&CK** framework mapping,
- **DevSecOps** pipeline integration.

No Breach also plays an active role in the broader cybersecurity ecosystem by organizing **hackathons** and **Capture The Flag (CTF)** competitions, giving students and enthusiasts the opportunity to test their skills in real-world scenarios, learn by doing, and connect with industry professionals.

3 Conclusion

To summarize, this chapter introduced the host organization, No Breach Training Hub, and its activities in cybersecurity. Understanding this environment highlights the importance of network segmentation and security control in modern IT infrastructures.

To address these technical challenges, our project focuses on implementing a secure, virtualized network environment managed by OPNsense. The next chapter will present the virtualization platform, tools, and system setup used to realize this architecture.

Chapter 2

Environment Setup & Tools

Before building the lab, all necessary tools were selected and installed. Each tool was chosen for a specific reason, either because it is the industry standard, or it is free and open-source, or it is the best fit for a virtualized home lab environment.

1 Virtualization Platform: VirtualBox

All components of this lab run as virtual machines on a single physical host, making the entire infrastructure reproducible, isolated, and cost-free.



Figure 2.1: VirtualBox Logo

VirtualBox is a free, open-source software that allows running multiple operating systems as virtual machines on a single physical computer.

In this project, it hosts all the virtual machines : **OPNsense**, **Kali Linux**, and **Ubuntu Server**, on the same host PC.

VirtualBox was chosen over VMware Workstation because it is completely free with no license required, and it supports all the network adapter types needed for this project : **Bridged**, **Host-Only**, and **NAT**.

2 Firewall: OPNsense



Figure 2.2: OPNsense Logo

OPNsense is a free, open-source firewall and routing platform built on FreeBSD. It provides a full web-based GUI to manage **firewall rules**, **NAT**, **VPN**, and **intrusion detection**, all in one place.

OPNsense was chosen over pfSense because it is more actively maintained, has a cleaner interface, and includes **Suricata** as a built-in plugin, which is the IDS/IPS engine used in Sprint 4.

3 Attacker Machine: Kali Linux



Figure 2.3: Kali Linux Logo

Kali Linux is a Linux distribution built specifically for penetration testing and security auditing. It comes pre-installed with hundreds of security tools, making it the standard choice for **simulating attacks** in cybersecurity labs.

In this project, Kali is used as the **external attacker** positioned on the WAN network, launching all three attacks : Nmap port scan, hping3 SYN flood, and Hydra brute-force, against the lab infrastructure.

4 Target Machine: Ubuntu Server



Figure 2.4: Ubuntu Server Logo

Ubuntu Server is a lightweight Linux distribution used to host services and applications. In this project, it acts as the **web server** deployed inside the DMZ zone, running a simple **Python HTTP server on port 80** to simulate a publicly accessible website.

Ubuntu Server was chosen because it is lightweight, easy to configure, and perfectly suited for running a minimal web server in a virtualized environment.

5 Network Diagram Tool: Draw.io



Figure 2.5: Draw.io Logo

Draw.io is a free, browser-based tool for creating diagrams and network topology maps. It requires no installation and allows exporting diagrams in multiple formats.

It was used in this project to design and document the complete network topology, showing how the WAN, LAN, and DMZ zones are connected through the OPNsense firewall.

Chapter 3

Phase I: Architecture and Secure Connectivity

In this phase, we will build the core network infrastructure of the lab.

Sprint 1 establishes the network segmentation and firewall configuration, creating three isolated zones : LAN, WAN, and DMZ , controlled by an OPNsense firewall.

Sprint 2 then introduces encrypted remote access through a WireGuard VPN tunnel, allowing secure administrative access to the internal network from the outside.

Together, these two sprints form the security foundation on which all subsequent phases are built.

1 Sprint 1: Network Segmentation and Firewall Configuration

1.1 Virtual Network Design

Before installing and configuring the firewall, the virtual network architecture was planned and designed. The lab environment is built around three distinct network zones, each serving a specific role:

- **LAN (Local Area Network):** The internal trusted network where the host PC resides.
- **WAN (Wide Area Network):** Simulates the external/internet-facing network, from which attacks and external access are initiated.
- **DMZ (Demilitarized Zone):** An isolated zone hosting the Ubuntu server, accessible from the WAN but strictly separated from the LAN.

This segmentation ensures that even if the DMZ is attacked, the internal LAN remains protected. All traffic between zones is controlled and filtered by the OPNsense firewall, which sits at the center of the architecture.

1.2 Firewall Installation

With the network architecture defined, the next step was to install the firewall that will enforce traffic control between the three zones. The firewall solution that was used was

OPNSense due to its **open-source nature, rich feature set, and enterprise-grade capabilities**.

It was installed as a dedicated virtual machine in VirtualBox and configured with three virtual network adapters, each mapped to one of the defined network zones : **WAN, LAN,** and **DMZ**.

1.2.1 OPNsense Installation

The OPNsense OS was downloaded in ISO format and installed within its own virtual machine on the Virtual Box application. The VM was configured with **3 network adapters**, each corresponding to one of the defined network zones.

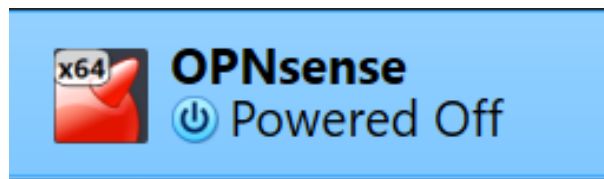


Figure 3.1: OPNsense VM in VirtualBox

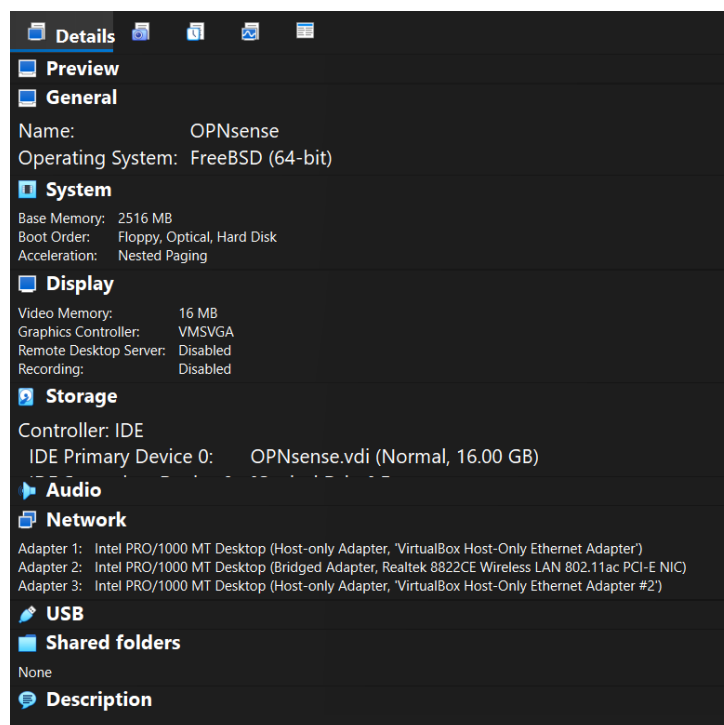


Figure 3.2: OPNsense VM Configuration : VirtualBox Details

The OPNsense virtual machine was configured with **2516 MB of RAM**, a **16 GB virtual disk**, and a **FreeBSD 64-bit operating system**.

1.2.2 Interface Assignment (WAN, LAN, and DMZ)

To bring this design to life, OPNsense was assigned three network adapters in VirtualBox, each mapped to a specific network zone:

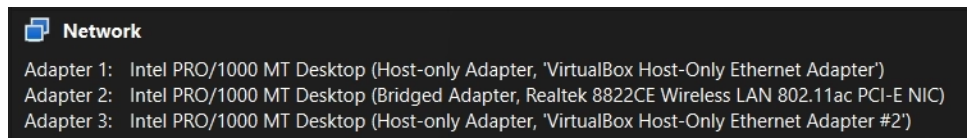


Figure 3.3: Network Adapter Configuration

- **Adapter 1 (Host-Only 1) : LAN** : provides connectivity to the internal network.
- **Adapter 2 (Bridged) : WAN** : simulates an external internet-facing connection through the host's physical NIC.
- **Adapter 3 (Host-Only 2) : DMZ** : isolates publicly accessible services from the internal LAN.

Each adapter was then assigned a static IP address to define the boundaries of its respective network zone.

1.2.3 IP Address Configuration (WAN, LAN, and DMZ)

The **LAN interface** (em0) was configured as the internal trusted network, allowing the host PC to communicate with the OPNsense firewall. em0 was configured directly from the OPNsense console, where the IP address and subnet were defined to establish the internal network boundary. To establish the LAN zone, a Host-Only network adapter was first created in VirtualBox and assigned to the LAN interface of the OPNsense VM.

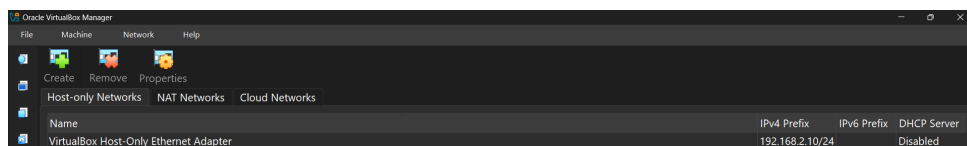


Figure 3.4: VirtualBox Network Manager : Host-Only Adapter

The VirtualBox Network Manager confirms the Host-Only adapter is configured with the prefix **192.168.2.10/24**, with the VirtualBox-level DHCP server disabled, address assignment is fully handled by OPNsense.

Once the adapter was ready, OPNsense was booted and the LAN IP address was assigned through the console setup wizard.

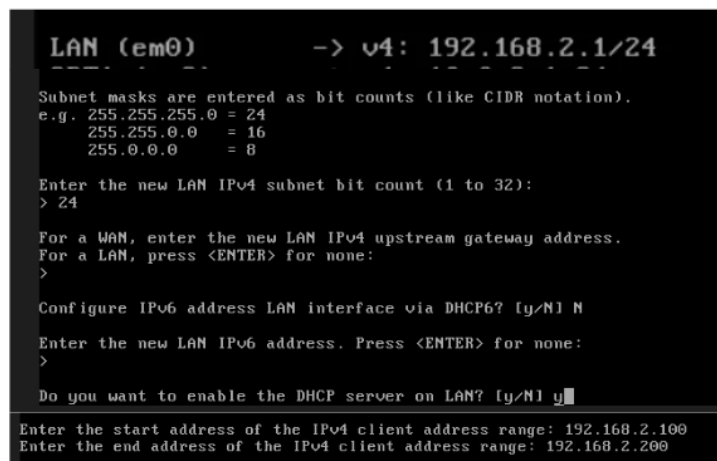


Figure 3.5: OPNsense Console: LAN IP Configuration

The LAN interface (em0) was assigned the static IP 192.168.2.1/24 directly from the OPNsense console. IPv6 was disabled, and DHCP was enabled with a range of 192.168.2.100 to 192.168.2.200, allowing the host PC to automatically receive an IP on the LAN network.

To avoid a conflict with the host machine's existing network, the LAN was placed on a separate subnet.

```

Ethernet adapter Ethernet 2:

Connection-specific DNS Suffix  . : 
Link-local IPv6 Address . . . . . : fe80::5628:9ef4:cc61:7401%9
IPv4 Address. . . . . : 192.168.1.1
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . . :

```

Figure 3.6: Avoiding IP Conflict

The host PC's Ethernet adapter already uses the 192.168.1.x subnet with IP 192.168.1.1. Since OPNsense's default LAN IP is also 192.168.1.1, keeping it would create a conflict. The LAN was therefore moved to 192.168.2.1/24, a completely separate subnet.

As a result, the host PC was automatically assigned an IP within the DHCP range, confirming successful communication with the firewall.

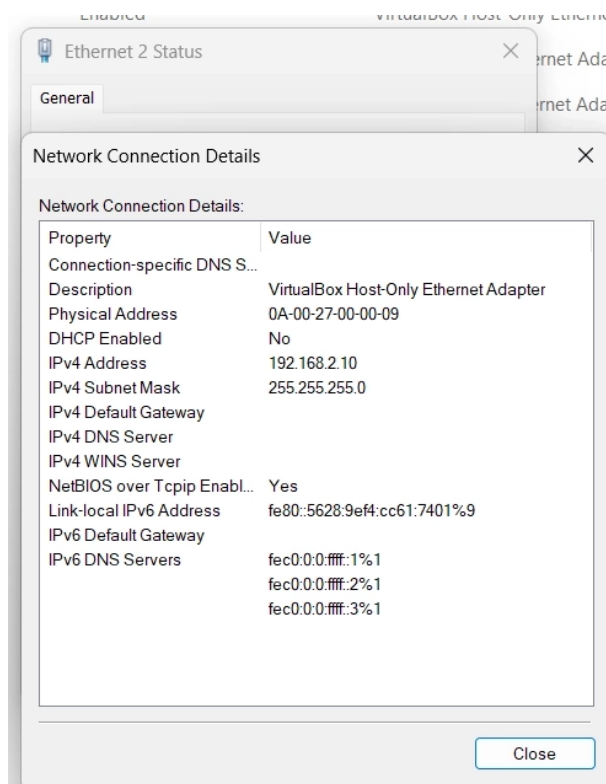


Figure 3.7: Host PC: Network Connection Details

The **host PC** received the IP **192.168.2.10** via DHCP from OPNsense, with a subnet mask of 255.255.255.0. This confirms that the host is correctly connected to the LAN zone and can reach the firewall at 192.168.2.1.

With the LAN interface configured, the **WAN interface** was set up to connect OPNsense to the **simulated WAN network** through the home router


```

Wireless LAN adapter Wi-Fi:

    Connection-specific DNS Suffix  . : home
    IPv6 Address. . . . .           : fd82:6fcf:7d12:bd00:a7b:e87c:1f6c:bf73
    Temporary IPv6 Address. . . . . : fd82:6fcf:7d12:bd00:d05e:4c1d:3f58:cf45
    Link-local IPv6 Address . . . . . : fe80::fa4a:f758:1a14:3396%8
    IPv4 Address. . . . .           : 192.168.1.148
    Subnet Mask . . . . .           : 255.255.255.0
    Default Gateway . . . . .       : 192.168.1.1

```

Figure 3.8: Host PC: Wi-Fi Adapter Configuration (ipconfig)

The **host PC's Wi-Fi adapter** operates on the **192.168.1.x subnet**, with IP 192.168.1.148 and a default gateway of 192.168.1.1. This means the WAN interface of OPNsense must be placed on the same subnet to reach the home router, which acts as the gateway of the simulated WAN network.

Since the home router uses the 192.168.1.x subnet, **the WAN interface (em1)** was assigned the static IP **192.168.1.2** to be on the same network.

A static IP was chosen instead of DHCP because VirtualBox Bridged mode over Wi-Fi often blocks DHCP requests, which would make the connection unreliable. With the static IP decided, the WAN interface was configured on OPNsense and verified through the console.

```

root@OPNsense:~ # ifconfig em1
em1: flags=1008843<UP,BROADCAST,RUNNING,SIMPLEX,MULTICAST,LOWER_UP> metric 0 mtu
1500
    description: WAN (wan)
    options=48500b8<VLAN_MTU,VLAN_HWTAGGING,JUMBO_MTU,VLAN_HWCSUM,VLAN_HWFIL
TER,VLAN_HWTSO,HWTSTATS,MEXTPG>
    ether 08:00:27:13:b6:3b
    inet 192.168.1.2 netmask 0xfffff00 broadcast 192.168.1.255
    media: Ethernet autoselect (1000baseT <full-duplex>)
    status: active
    nd6 options=29<PERFORMNUD,IFDISABLED,AUTO_LINKLOCAL>
root@OPNsense:~ #

```

Figure 3.9: OPNsense Console: WAN Interface Verification (ifconfig em1)

The ifconfig output confirms that the WAN interface (em1) is active and correctly assigned the static IP 192.168.1.2 with a subnet mask of 255.255.255.0. The interface is running in full-duplex Gigabit Ethernet mode (1000baseT), indicating that the simulated WAN connection is active and operating correctly.

Note: In this lab, the WAN is simulated using the private subnet 192.168.1.0/24. The home router acts as the simulated internet gateway, and Kali Linux acts as a simulated external attacker, replicating real-world WAN behavior within a fully virtualized environment.

With both the LAN and WAN interfaces configured, **the DMZ** was set up as a completely isolated zone to host publicly accessible services, separate from the internal network.

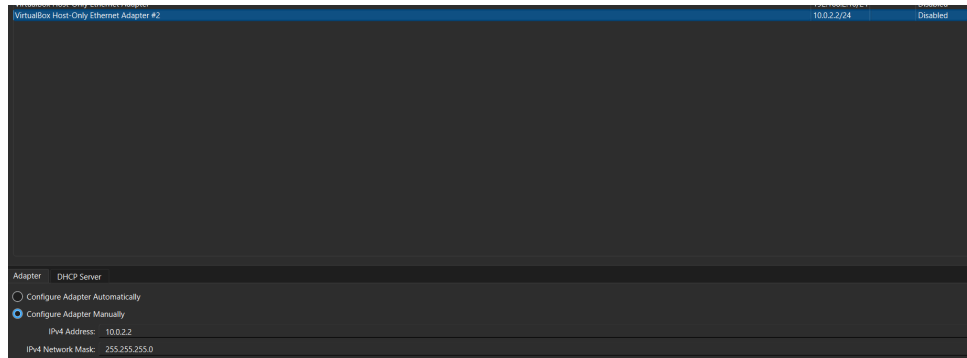


Figure 3.10: VirtualBox Network Manager: DMZ Host-Only Adapter 2

A **second Host-Only adapter** was created in VirtualBox and manually configured **with the IP 10.0.2.2/24**. The DMZ was placed on the 10.0.2.x subnet to keep it fully isolated from both the LAN (192.168.2.x) and the WAN (192.168.1.x).

The 10.0.2.x range was chosen specifically to avoid conflicts with other private services such as VPN tunnels or Docker containers that commonly use 10.0.0.x. DHCP was disabled at the VirtualBox level since the only device in the DMZ (the Ubuntu server) will be assigned a static IP manually.

The OPNsense DMZ interface (em2) at 10.0.2.1 acts as **the gateway for the DMZ network**, ensuring that all traffic entering or leaving the DMZ passes through the firewall and remains isolated from the internal LAN.

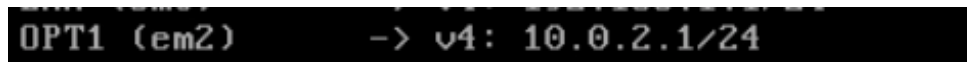


Figure 3.11: OPNsense Console: DMZ Interface Assignment (em2)

The OPNsense console confirms that the DMZ interface (em2), labeled OPT1, was successfully assigned the IP 10.0.2.1/24, acting as the gateway for the DMZ network where the Ubuntu server will be hosted. To verify the network from the host side, the DMZ adapter details were checked on the host PC. The Host-Only adapter was manually set to 10.0.2.2, a separate address on the same DMZ subnet, distinct from the OPNsense gateway IP (10.0.2.1). This confirms both interfaces coexist on the network without any IP conflict.

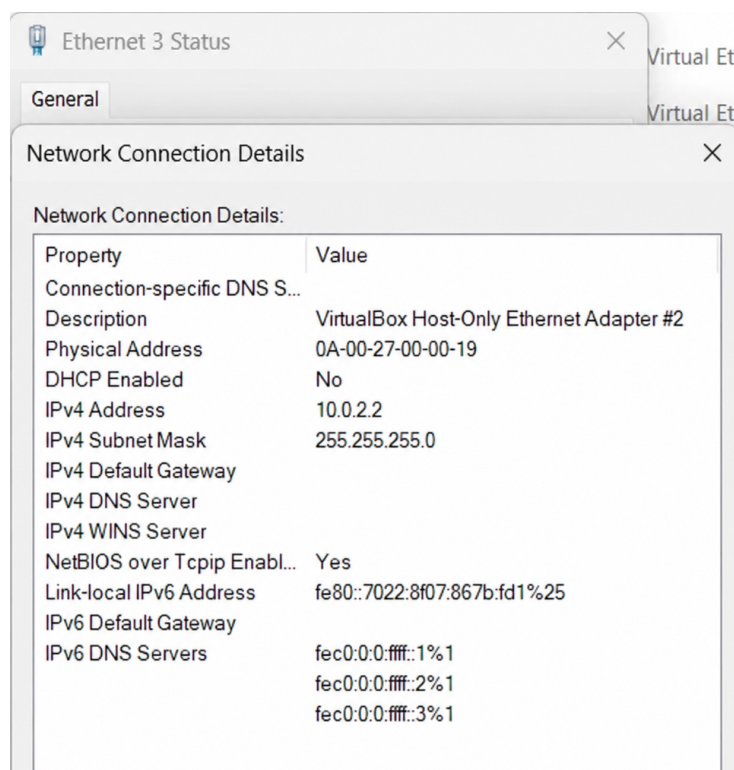


Figure 3.12: Host PC : VirtualBox Host-Only Adapter 2 configuration (10.0.2.2)

The host PC's Ethernet 3 adapter, mapped to VirtualBox Host-Only Ethernet Adapter 2, confirms the IP 10.0.2.2 with subnet mask 255.255.255.0. DHCP is disabled, as address assignment in the DMZ will be handled statically for the Ubuntu server.

With all three interfaces configured, OPNsense displays the final interface assignment summary confirming the complete network architecture.

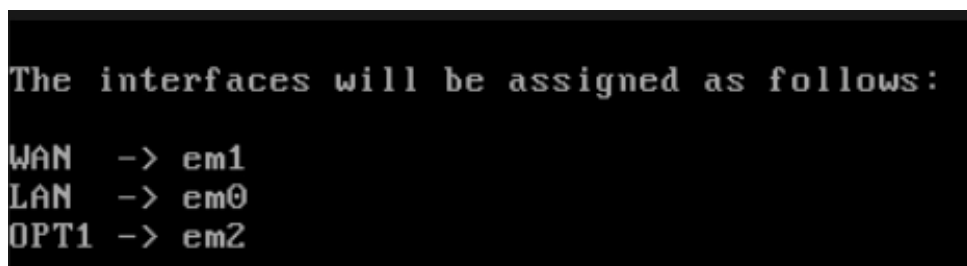


Figure 3.13: OPNsense Interface Assignment Summary

Upon completing the configuration, OPNsense confirmed the final mapping of all three network interfaces: WAN (em1), LAN (em0), and OPT1 (DMZ). This assignment ties each physical virtual adapter to its corresponding network zone, forming the foundation of the segmented lab architecture.

The following diagram summarizes the complete **network topology of the lab**, illustrating how all three zones are interconnected through the OPNsense firewall:

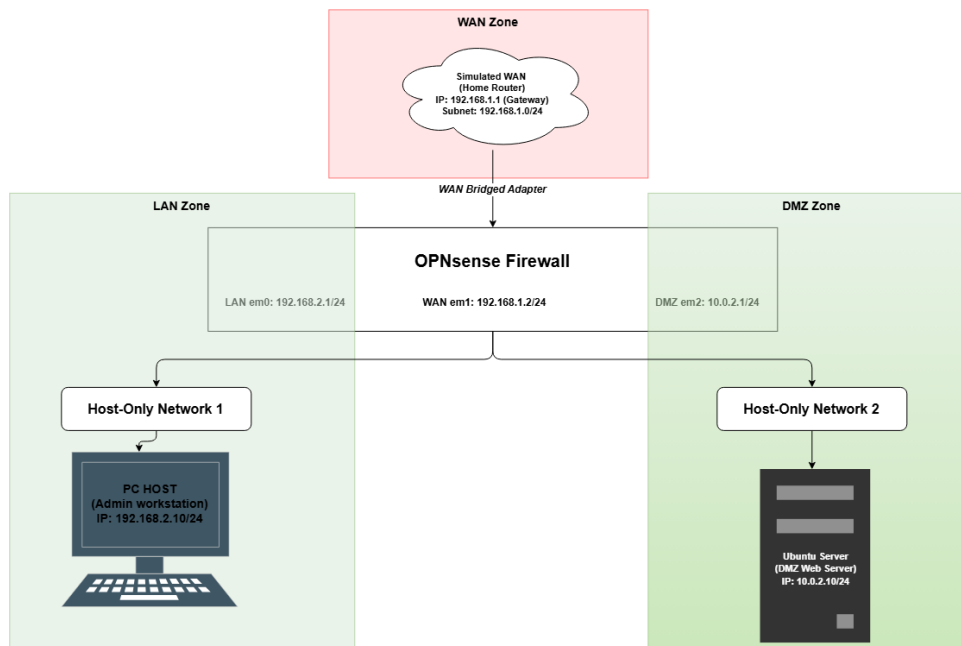


Figure 3.14: Lab Network Topology

The topology places OPNsense at the center of the architecture, acting as the traffic controller between all three zones.

- **WAN Interface (em1):** Configured in **Bridged mode**. It receives the IP 192.168.1.2/24 and uses the home router (192.168.1.1) as its **Default Gateway**.
- **LAN Interface (em0):** Assigned 192.168.2.1/24. It connects to the **Host PC** (192.168.2.10) through the first **Host-Only Adapter**.
- **DMZ Interface (em2):** Assigned 10.0.2.1/24. It connects to the **Ubuntu Server** (10.0.2.10) through the second **Host-Only Adapter** to isolate services from the LAN.

With all interfaces configured, the following steps confirm that OPNsense is fully operational and accessible from the host PC.

```
C:\Users\meriem pc>ping 192.168.2.1

Pinging 192.168.2.1 with 32 bytes of data:
Reply from 192.168.2.1: bytes=32 time=2ms TTL=64
Reply from 192.168.2.1: bytes=32 time=2ms TTL=64
Reply from 192.168.2.1: bytes=32 time=2ms TTL=64
Reply from 192.168.2.1: bytes=32 time=2ms TTL=64

Ping statistics for 192.168.2.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 2ms, Maximum = 2ms, Average = 2ms
```

Figure 3.15: LAN Connectivity Test (ping 192.168.2.1)

The ping test confirmed basic connectivity, and the OPNsense web GUI was then accessed to verify full management access.

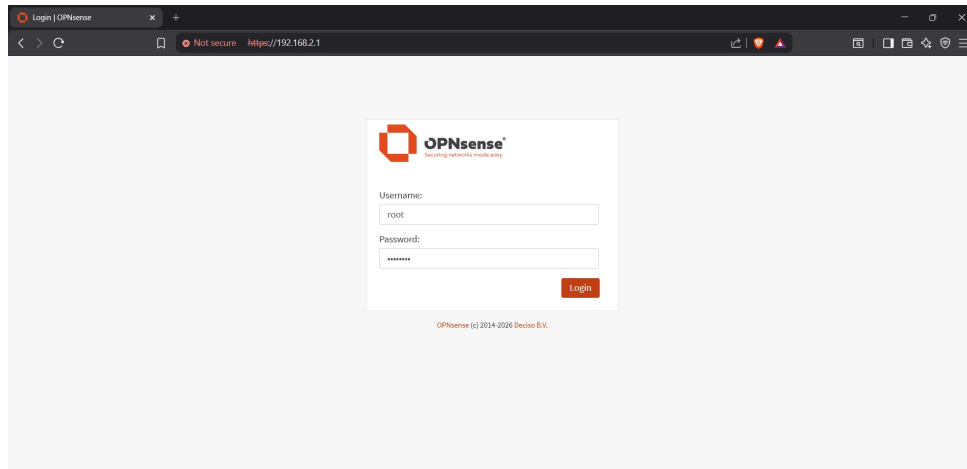


Figure 3.16: OPNsense Web GUI Login Page

The OPNsense web interface was accessed from the host PC's browser at **https://192.168.2.1**. The login page loaded successfully, confirming that the firewall is reachable and the web management interface is active.

The OPNsense Web GUI is accessed **via the LAN interface IP (192.168.2.1)**, which hosts the firewall's management web server. The host PC (192.168.2.10) acts as **the client** requesting access from the trusted LAN segment, where OPNsense permits administrative traffic while blocking management access from the WAN by default for security.

After logging in, the OPNsense configuration wizard confirmed the system is ready for further configuration.

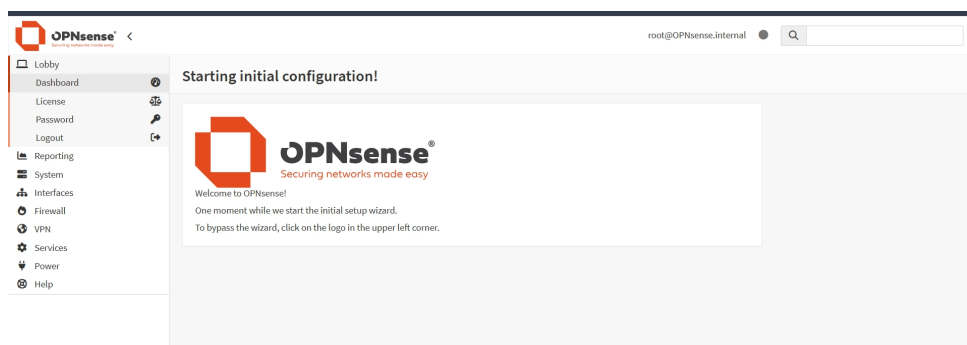


Figure 3.17: OPNsense Dashboard : Initial Configuration Screen

Upon logging in, the OPNsense dashboard displayed the initial configuration screen, with the full navigation menu visible including Firewall, VPN, Services and Interfaces. This confirms that the firewall is fully operational and ready for the next configuration steps.

1.3 Access Control Lists (ACLs)

With the network interfaces fully configured, firewall rules were defined to control traffic between the three network zones. The ACL policy enforces two main rules:

- **LAN to WAN (Internet Access):** Allows outbound traffic from the LAN (192.168.2.0/24) to the WAN so the host PC can access the internet.
- **DMZ to LAN (Block Access):** Blocks traffic originating from the DMZ (10.0.2.0/24) to the LAN to protect internal machines.

To enforce these security boundaries, dedicated firewall rule sets were configured on both the LAN and DMZ interfaces.

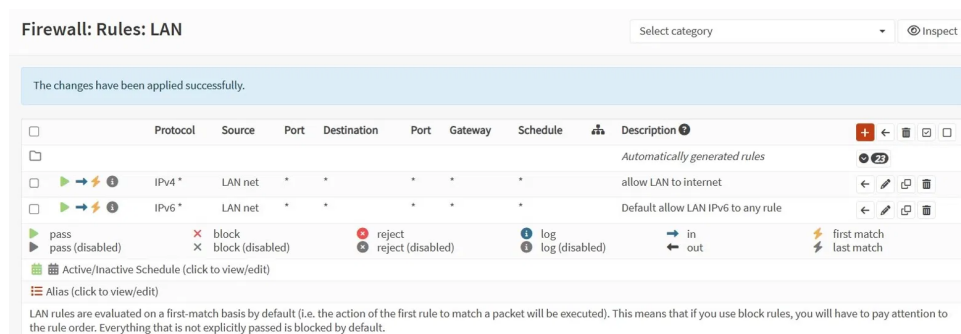


Figure 3.18: Firewall Rules: LAN Interface

A rule was configured on the LAN interface that lets all IPv4 traffic from the local area network (192.168.2.x) go to any place it wants. This means the host PC (192.168.2.10) can use the internet without any problems. This rule is called **”allow LAN to internet”** and it works for all ports and protocols so the internal local area network has full access, to the outside.

To enforce isolation between the DMZ and the internal network, a dedicated block rule was added on the DMZ interface.

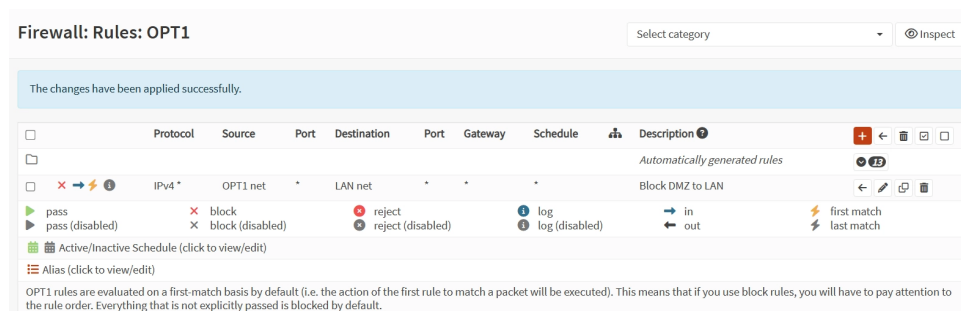


Figure 3.19: Firewall Rules: DMZ Interface (OPT1)

A block rule was added on the DMZ interface (OPT1), preventing any traffic **from the DMZ network from reaching the LAN**. This means that even if the Ubuntu server is breached, the attacker cannot access the internal network, the breach stays contained within the DMZ.

1.4 NAT and Port Forwarding

NAT Port Forwarding redirects traffic arriving at the WAN interface (which acts as the firewall’s simulated external address) to a specific machine in the DMZ. In this setup, we

configure the firewall: *"When external visitors request port 80 (HTTP) on our WAN IP, forward them directly to the Ubuntu server (10.0.2.10)."*

This creates a path allowing external clients on the internet on the simulated WAN to **securely access the web service** that our Ubuntu server hosts (running Python HTTP server on port 80).

To achieve this, an Ubuntu server was first installed and configured in the DMZ as the target of the port forwarding rule.

1.4.1 Ubuntu Server Installation & Configuring

The Ubuntu Server VM (chosen for its lightweight resource usage and strong community support) was created in VirtualBox, attached to the DMZ network, and configured with a static IP address to act as the target web server.



Figure 3.20: Ubuntu Server VM in VirtualBox

The Ubuntu Server was installed as a dedicated virtual machine in VirtualBox, named "Ubuntu Server"

To place the Ubuntu server in the DMZ zone, its network adapter was attached to the same Host-Only adapter used by the OPNsense DMZ interface.

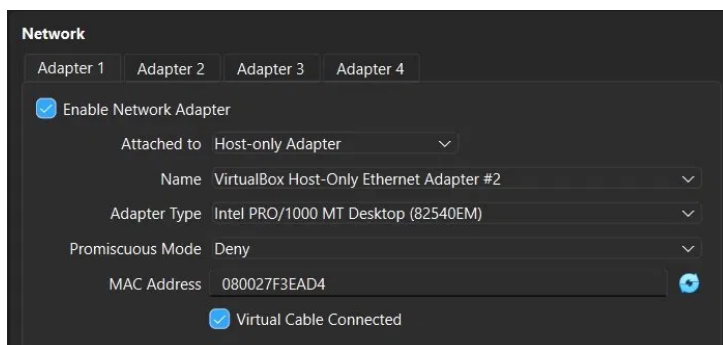


Figure 3.21: Ubuntu Server Network Adapter Configuration

The Ubuntu Server's network adapter was attached to VirtualBox Host-Only Ethernet Adapter 2, the same adapter used by the OPNsense DMZ interface (em2). This places the server directly inside the DMZ network, isolated from the LAN.

Once the adapter was connected, a static IP was manually assigned to the server during the Ubuntu installation process.

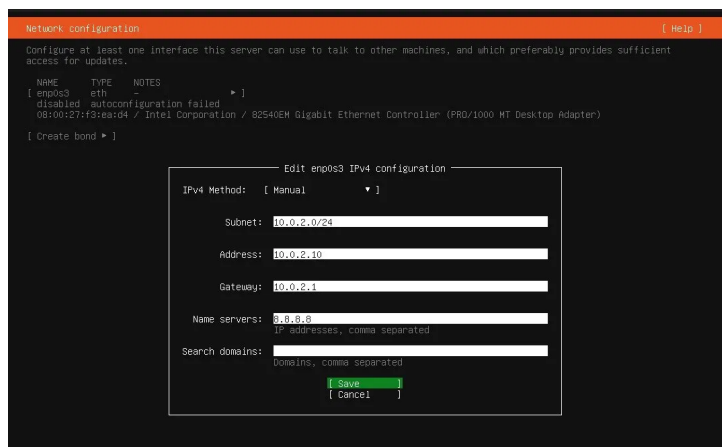


Figure 3.22: Ubuntu Server Static IP Configuration

During installation, the primary network interface (`enp0s3`, the server's virtual network adapter) was configured with the static IP `10.0.2.10/24` and default gateway `10.0.2.1` (OPNsense). A static IP was necessary because DHCP is disabled in the DMZ zone. Additionally, Google's public DNS (`8.8.8.8`) was selected to guarantee reliable external name resolution, as no local DNS server exists in this lab setup.

During installation, the server profile was configured with a username and hostname to identify the machine within the network.



Figure 3.23: Profile Configuration

The server was configured with the hostname "**ubuntu-dmz**", making it easily identifiable on the network, and a user account was created for system access. This naming convention clearly reflects the server's role and location within the DMZ zone.

After installation, the IP configuration was verified directly from the Ubuntu terminal.

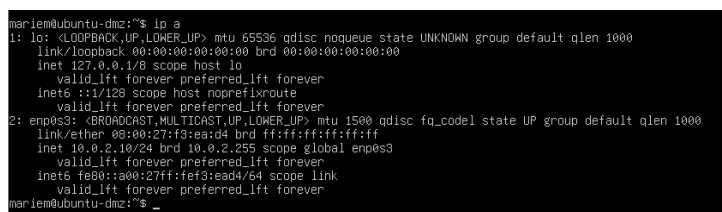


Figure 3.24: Ubuntu Server IP Verification (ip a)

The `ip a` command confirms that the network interface `enp0s3` is active and correctly assigned the static IP `10.0.2.10/24`, verifying that the server is properly configured and ready to receive forwarded traffic from OPNsense.

1.4.2 Launching the Python Web Server

To simulate a public web server in the DMZ, a lightweight **Python HTTP server** (chosen over heavy web servers like Apache or Nginx for its instant setup with no extra installation or configuration needed) was launched on the Ubuntu server, listening on port 80 and **ready to receive HTTP requests forwarded from OPNsense**.

```
mariem@ubuntu-dmz:~$ sudo python3 -m http.server 80
[sudo] password for mariem:
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

Figure 3.25: Python HTTP server running on Ubuntu in the DMZ

The command **sudo python3 -m http.server 80** was executed on the Ubuntu server, starting a lightweight HTTP server listening on all interfaces (0.0.0.0:80).

This means the server is ready to accept incoming HTTP requests on port 80 from any source, making it the target for the NAT port forwarding rule that will be configured next.

1.4.3 Creating a Port Forwarding Rule

With the Python web server running on the Ubuntu server, a NAT port forwarding rule was created on OPNsense to redirect incoming HTTP traffic from the WAN to the Ubuntu server in the DMZ.

Categories	Version	Protocol	Source	Port	Destination	Port	Redirect Tar...	Redirect ...	Description
☐ LAN	*	TCP	*	*	LAN address	443	*	*	Anti-Lockout Rule
☐ LAN	*	TCP	*	*	LAN address	80	*	*	Anti-Lockout Rule
☒ WAN	IPv4	TCP	*	*	WAN address	http	10.0.2.10	http	Port forward HTTP to Ubuntu DMZ

Figure 3.26: OPNsense NAT : Destination NAT Rule

A port forwarding rule was configured on the WAN interface, redirecting all incoming IPv4 TCP traffic arriving on port 80 (HTTP) from any source to the Ubuntu server at 10.0.2.10:80 in the DMZ.

This rule tells OPNsense to forward any external HTTP request received on the WAN address directly to the Ubuntu web server, making it accessible from the outside network including the Kali Linux VM.

1.5 Sprint 1 Validation and Testing

To validate the entire Sprint 1 configuration, two tests were conducted to verify that the firewall rules and port forwarding are working as expected.

1.5.1 Test 1: Web Server Accessibility from Kali Linux (WAN Side)

Before testing, a WAN firewall rule was verified to ensure incoming HTTP traffic is permitted through the firewall.

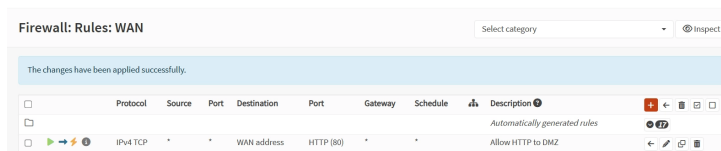


Figure 3.27: WAN Firewall Rule: Allow HTTP to DMZ

The WAN firewall rule confirms that all incoming IPv4 TCP traffic on port 80 (HTTP) from any source to the WAN address is allowed. This rule works together with the NAT port forwarding rule to deliver the traffic to the Ubuntu server at 10.0.2.10 in the DMZ.

To confirm the Kali VM is correctly positioned on the WAN network before running the test, its IP address was verified.

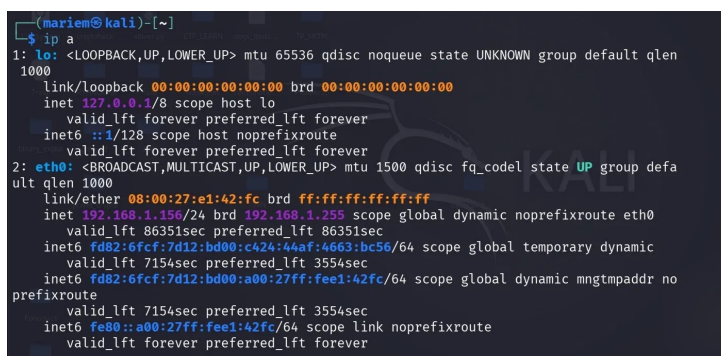


Figure 3.28: Kali Linux IP Configuration (ip a)

The Kali VM's network interface eth0 is assigned the IP 192.168.1.156/24, placing it on the 192.168.1.x WAN subnet. This confirms that Kali is on **the external network** and will reach the Ubuntu server exclusively through the OPNsense port forwarding rule.

With everything in place, an HTTP request was sent from Kali to the OPNsense WAN IP to test the port forwarding.

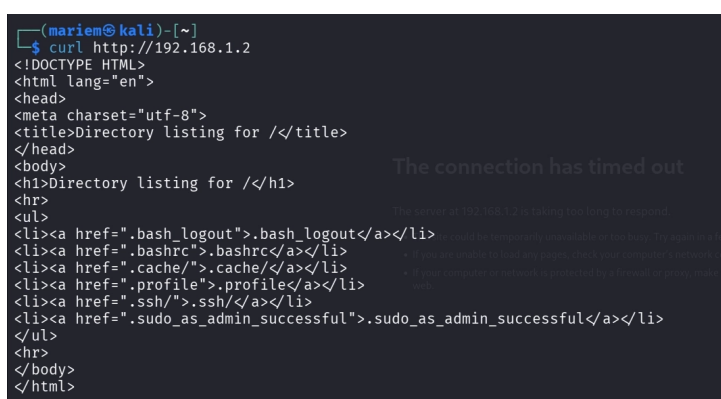
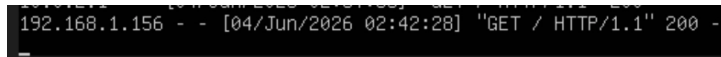


Figure 3.29: HTTP Request from Kali to Ubuntu Web Server (curl)

The command `curl http://192.168.1.2` was executed from Kali, targeting the OPNsense WAN IP. The Ubuntu server's directory listing was returned successfully in HTML format, confirming that OPNsense correctly forwarded the request from the WAN interface to the Ubuntu server in the DMZ. The Ubuntu server logs were then checked to confirm the request was received.



```
192.168.1.156 - - [04/Jun/2026 02:42:28] "GET / HTTP/1.1" 200 -
```

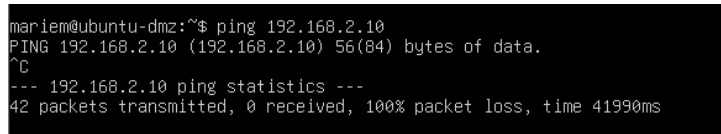
Figure 3.30: Ubuntu Server HTTP Request Log

The Ubuntu server logs confirm that an HTTP GET request was received from 192.168.1.156 (Kali VM) with a 200 OK response code.

This proves that the port forwarding rule successfully delivered the external traffic to the DMZ web server.

1.5.2 Test 2: DMZ-to-LAN Isolation Verification

To confirm that the DMZ is fully isolated from the internal network, a ping was launched from the Ubuntu server in the DMZ toward the host PC on the LAN.



```
marjem@ubuntu-dmz:~$ ping 192.168.2.10
PING 192.168.2.10 (192.168.2.10) 56(84) bytes of data:
^C
--- 192.168.2.10 ping statistics ---
42 packets transmitted, 0 received, 100% packet loss, time 41990ms
```

Figure 3.31: Ubuntu DMZ Server : Ping to LAN (192.168.2.10)

A ping from the Ubuntu server (10.0.2.10) to the host PC (192.168.2.10) resulted in 100% packet loss. This confirms that the OPNsense block rule on the DMZ interface is working correctly, preventing any traffic initiated from the DMZ from reaching the LAN. This is a critical security validation: even if an attacker compromises the Ubuntu web server, they cannot reach the internal network.

1.6 Bonus: Python-Based Reachability Scanner

Instead of manually testing each firewall rule with ping or curl commands, a Python script was developed to automatically validate all ACL rules across the three network zones and generate a clear Pass/Fail report.

This script uses **the socket library** (The tool that creates the network connections to test or check if a specific port on a target IP address is open and listening for connections) to attempt TCP connections to specific *IP:Port* targets and reports whether each connection succeeded or failed as expected.

It was **executed on the Kali VM** (WAN side : 192.168.1.156) because Kali simulates an external attacker trying to reach different network zones, making it the most realistic machine to validate the firewall rules from the outside.

Before running the script, two services were started: OPNsense was already running, and the Ubuntu server had the Python HTTP server active on port 80 with **sudo python3 -m http.server 80**. The script was designed to run three tests from the Kali VM:

- **WAN → DMZ web server (port 80):** Expected to **PASS** because the port forwarding rule redirects incoming HTTP traffic to the Ubuntu server.
- **WAN → LAN PC host (port 80):** Expected to **FAIL** because no rule allows WAN traffic to reach the LAN network.
- **WAN → OPNsense LAN GUI (port 443):** Expected to **FAIL** because the web GUI is restricted to LAN access only.

With the test plan defined, the script was executed on Kali and the results were recorded.

```
(mariem@kali)-[~/Desktop/script_p]
$ python3 reachability_scanner.py

=====
FIREWALL REACHABILITY SCANNER
Running from: Kali VM (WAN - 192.168.1.156)
=====

Test : WAN → DMZ Web Server (port forwarding)
Target : 192.168.1.2:80
Expected: PASS | Got: PASS | [✓ OK ]

Test : WAN → LAN PC Host
Target : 192.168.2.10:80
Expected: FAIL | Got: FAIL | [✓ OK ]

Test : WAN → OPNsense LAN GUI
Target : 192.168.2.1:443
Expected: FAIL | Got: FAIL | [✓ OK ]

=====
RESULTS - Passed: 3/3 | Failed: 0/3
✓ All firewall rules are working correctly!
=====
```

Figure 3.32: Python Reachability Scanner Output

The scanner was executed from Kali with the command **python3 reachability-scanner.py**. All three tests produced the expected results : **3/3 passed with 0 failures**, confirming that all firewall rules are working correctly : The DMZ web server is reachable from WAN via port forwarding, while both LAN access attempts were correctly blocked by the firewall.

1.7 Conclusion

Sprint 1 successfully laid the foundation of the lab. Starting from scratch, we designed and built a three-zone network : **WAN, LAN, and DMZ** , controlled by an OPNsense firewall.

We configured the three network interfaces, assigned static IP addresses to each zone, and set up firewall ACL rules to allow the LAN full internet access while completely blocking the DMZ from reaching the internal network.

We then deployed an Ubuntu web server inside the DMZ, launched a Python HTTP server on it, and configured NAT port forwarding so that external HTTP traffic arriving on the WAN is automatically redirected to the Ubuntu server.

Finally, everything was validated with a Python reachability scanner that confirmed all firewall rules are working correctly : the DMZ web server is reachable from outside, and the internal LAN remains fully protected.

2 Sprint 2: Encrypted VPN Tunneling

With the network segmentation and firewall rules established in Sprint 1, the next challenge is **secure remote access**.

A **VPN (Virtual Private Network)** is an encrypted tunnel that allows a remote device to securely join a private network over an untrusted network, as if it were physically connected to it.

Sprint 2 introduces a **WireGuard tunnel** (a lightweight VPN protocol that creates fast, encrypted connections) that gives the remote administrator (Kali) secure and authenticated access to the internal LAN, including the OPNsense management GUI, without opening unnecessary firewall rules.

So the Kali VM will now be able to **open the OPNsense firewall GUI** and **fully access all internal resources within the LAN network**.

The purpose of giving the Kali VM access to the internal LAN is to:

- **Secure Remote Administration:** Allowing the administrator to securely access the OPNsense management GUI from the WAN side, which is completely blocked without the VPN.
- **Simulate Real-World Remote Access:** In a real network, the firewall is never managed directly from the internet. The VPN ensures that only authenticated users can get in.
- **Avoid Exposing the Management Interface:** Without the VPN, the only way to reach OPNsense from outside would be to open a port forwarding rule on the WAN, which would expose the management interface to everyone. The VPN eliminates that risk entirely.

2.1 WireGuard Overview & Availability

OPNsense supports multiple VPN protocols including IPsec, OpenVPN, and WireGuard.

WireGuard was selected for its simplicity, speed, and modern cryptography with minimal configuration overhead. It comes built into OPNsense and requires no additional installation, it is available directly from the VPN menu as shown below:

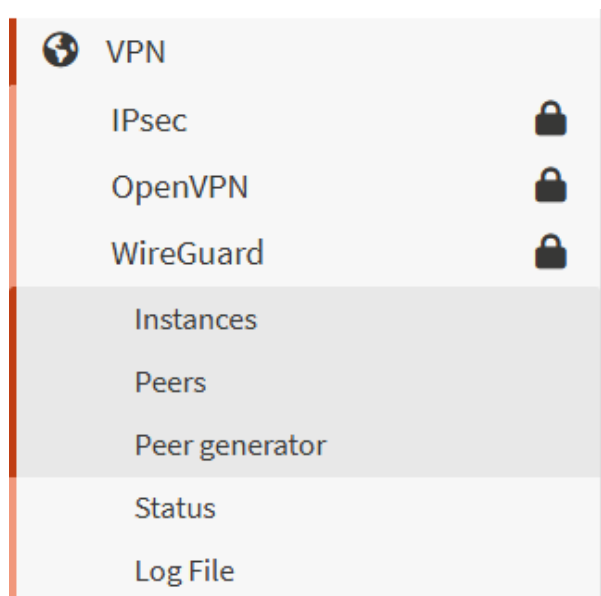


Figure 3.33: OPNsense VPN Menu: WireGuard Available

The OPNsense VPN menu confirms that WireGuard is readily available, exposing the Instances, Peers, Peer Generator, Status, and Log File sections needed for tunnel configuration.

2.2 Cryptographic Key Generation

WireGuard uses **public/private key pairs** to authenticate and encrypt the VPN tunnel instead of passwords, each machine proves its identity using cryptography. Because WireGuard requires mutual authentication, each side must know the other's public key before the tunnel can be established.

To achieve this, a key pair must be generated for both endpoints:

- **The VPN Server (OPNsense firewall):** Generates its own key pair and acts as the **central gateway** (because it is the single entrance that checks external users and connects them to the internal network.). It must know the Kali VM's public key to authenticate it before establishing the tunnel.
- **The VPN Client (Kali VM):** Generates its own key pair and acts as the **remote device** (because it is located outside the private network: on the public WAN side). It must know the OPNsense firewall's public key to verify its identity before connecting.

2.2.1 OPNsense Key Generation

OPNsense **automatically generates the WireGuard key pair** through its web interface. The private key remains stored securely on OPNsense and is never shared, while the public key will be shared with the Kali client to authenticate the tunnel.

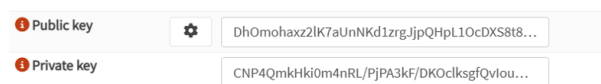


Figure 3.34: OPNsense Key Generation: Auto-generated WireGuard Key Pair

Since both ends of the tunnel must authenticate each other, a key pair was also generated on the Kali client.

2.2.2 Kali Key Generation

The private key was generated using **wg genkey** (the standard command line utility used to create a secure, random private key) and saved to **kali-private.key**. The corresponding public key was then derived from it using **wg pubkey** (the command that calculates the matching public key from an existing private key) and saved to **kali-public.key**. This public key will be shared with OPNsense to authenticate the Kali client on the VPN tunnel.

```

--(mariem@kali)-[~/Desktop/wireguard-keys]
-$ wg genkey | tee kali_private.key
NAULFrNJUnicW7M07qX3nUH08/m6DzD9Cd3$fu27U9Y/=
--(mariem@kali)-[~/Desktop/wireguard-keys]
-$ cat kali_private.key | wg pubkey | tee kali_public.key
PH2cYHKC7nqZekoPkw1cnXBZooxWpTVnW6+HmoaWyM=

```

Figure 3.35: Kali Key Generation:WireGuard Key Pair Generated on Kali

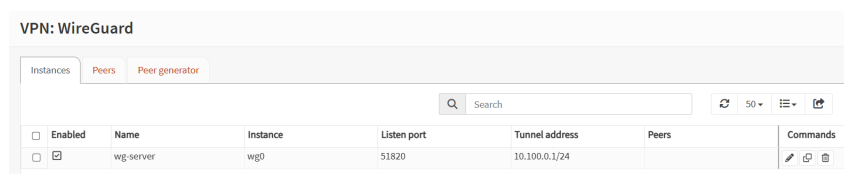
2.3 Tunnel Configuration

With the key pairs generated, the **WireGuard tunnel**(a secure, encrypted pathway created over an untrusted network to safely transmit data between two endpoints) was configured on OPNsense. This was done by defining the **VPN subnet** (a dedicated, isolated IP address range reserved exclusively for the virtual tunnel interfaces) and **opening the necessary firewall port** to allow the external Kali client to establish its connection.

2.3.1 Tunnel Subnet

A **WireGuard server instance** was created on OPNsense, defining the VPN tunnel network and the listening port for incoming client connections.

This instance acts as the listening host that accepts incoming traffic from the external Kali client and securely bridges it into the internal network zones.



The screenshot shows the 'VPN: WireGuard' configuration page in OPNsense. It has tabs for 'Instances', 'Peers', and 'Peer generator'. The 'Instances' tab is active, displaying a table with one instance named 'wg-server'. The table columns are: Enabled (checkbox), Name, Instance, Listen port, Tunnel address, Peers, and Commands.

Enabled	Name	Instance	Listen port	Tunnel address	Peers	Commands
☒	wg-server	wg0	51820	10.100.0.1/24		

Figure 3.36: WireGuard Server Instance on OPNsense

A server instance named **wg-server** was created on interface **wg0**, listening on **UDP port 51820** with a tunnel address of 10.100.0.1/24.

The subnet 10.100.0.0/24 was chosen because it does not conflict with any existing network in the lab : LAN (192.168.2.x) and DMZ (10.0.2.x) creating a dedicated isolated subnet exclusively for the VPN tunnel.

Port 51820 is the official default WireGuard port, following standard convention just like HTTP uses port 80.

To allow external VPN traffic to reach the WireGuard server, a firewall rule was added on the WAN interface.

2.3.2 WAN Firewall Rule : Allow WireGuard UDP 51820

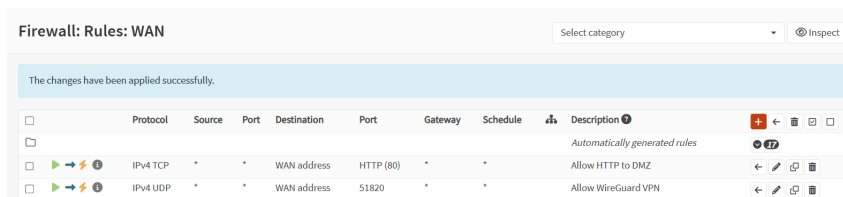


Figure 3.37: WAN Firewall Rule: Allow WireGuard VPN

A new rule was added on the WAN interface allowing all incoming **IPv4 UDP** traffic on port 51820 **from any source to the WAN address**. This rule opens the WireGuard port on the firewall, enabling the Kali client to reach the OPNsense VPN server from the external network.

2.4 Client Provisioning & Verification

This section proves that **the VPN is fully operational** by connecting Kali (WAN side) through the encrypted WireGuard tunnel into the internal LAN, allowing it to securely access the OPNsense management GUI , which is normally unreachable from the WAN.

Before this task, Kali was an outsider : it could only reach the DMZ web server via port forwarding. After this task, Kali becomes a **trusted remote administrator**, able to securely manage OPNsense from the WAN side as if it were physically inside the LAN.

2.4.1 Kali Peer Registration on OPNsense

Before establishing the tunnel, OPNsense must know who Kali is , this is done by registering **Kali as a trusted peer**.

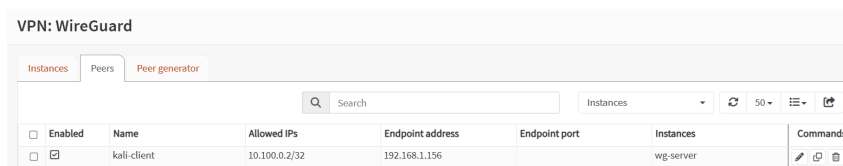


Figure 3.38: WireGuard Peer: Kali Client Registered on OPNsense

The Kali client was registered as a peer named **kali-client** on the wg-server instance, with endpoint address 192.168.1.156 and allowed IP 10.100.0.2/32. This address serves as **Kali's private identity inside the virtual tunnel**, completely separate from its public external IP. By putting this address in the **AllowedIPs** field, the OPNsense server mathematically locks this tunnel IP directly to Kali's public cryptographic key.

This configuration ensures that only Kali can use this address, allowing OPNsense to know exactly which authenticated peer is sending and receiving traffic through this specific connection.

2.4.2 Enabling WireGuard on OPNsense

With the peer registered, the WireGuard service was activated on OPNsense to start listening for incoming client connections.

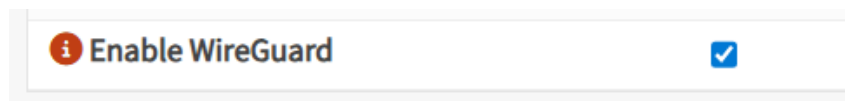


Figure 3.39: WireGuard Service Enabled on OPNsense

WireGuard was enabled on OPNsense, activating the VPN server and making it listen on UDP port 51820 for incoming encrypted connections from Kali.

2.4.3 Kali Client Configuration

With OPNsense ready, the **WireGuard configuration file** was created on Kali, providing all the necessary parameters **to establish the encrypted tunnel**.

```
(mariem@kali)-[~]
$ sudo cat /etc/wireguard/wg0.conf
[Interface]
PrivateKey = 4NAULFrNJUnicW7M07qX3nUH08/m6DxDECd58u27U0Y=
Address = 10.100.0.2/24
DNS = 192.168.2.1

[Peer]
PublicKey = Dh0mohaxz2lK7aUnNKd1zrgJjpQHPL10cDXS8t8KCwo=
Endpoint = 192.168.1.2:51820
AllowedIPs = 10.100.0.0/24, 192.168.2.0/24
PersistentKeepalive = 25s
```

Figure 3.40: WireGuard Configuration File on Kali (wg0.conf)

The configuration file defines two sections:

- **the Interface section** sets Kali's private key, VPN IP address 10.100.0.2/24, and DNS server 192.168.2.1 (the OPNsense LAN interface, which acts as the DNS resolver for devices on the internal network).
- **The Peer section** points to OPNsense using its public key, endpoint 192.168.1.2:51820, and allowed IPs 10.100.0.0/24 and 192.168.2.0/24, giving Kali access to both the VPN tunnel subnet and the internal LAN.

Each machine sees the other as its peer, they are simply two sides of the same encrypted tunnel.

To secure the configuration file, the correct permissions were applied before activating the tunnel.

```
(mariem@kali)-[~]
$ sudo chmod 600 /etc/wireguard/wg0.conf
[sudo] password for mariem:
```

Figure 3.41: Setting File Permissions on wg0.conf

The command **sudo chmod 600 /etc/wireguard/wg0.conf** was executed to restrict access to the configuration file, ensuring that only the root user can read it, protecting the private key stored inside.

2.4.4 Connecting Kali to the VPN

With the configuration secured, the WireGuard tunnel was activated on Kali. Note that the **resolvconf** package was first installed using **sudo apt install resolvconf -y** to handle DNS resolution through the tunnel (without it, Kali would stay using its default public DNS and would be unable to resolve internal hostnames even after connecting to the VPN).

```
(mariem@kali)-[~]
$ sudo wg-quick up wg0
[#] ip link add dev wg0 type wireguard
[#] wg setconf wg0 /dev/fd/63
[#] ip -4 address add 10.100.0.2/24 dev wg0
[#] ip link set mtu 1420 up dev wg0
[#] resolvconf -a tun.wg0 -m 0 -x
[#] ip -4 route add 192.168.2.0/24 dev wg0
```

Figure 3.42: Connecting Kali to the VPN

The command **sudo wg-quick up wg0** activated the VPN tunnel : Kali was assigned the VPN IP 10.100.0.2 and can now reach the internal LAN (192.168.2.0/24) through the encrypted connection.

2.5 Verify the Connection

With the tunnel activated, the connection was verified from both the Kali and OPNsense sides to confirm that the encrypted VPN is fully operational.

```
(mariem@kali)-[~]
$ sudo wg show
interface: wg0
  public key: rPH2cYHKC7nqZgkoPkw1cnXBZooxWpTVnW6+HmoaWyM=
  private key: (hidden)
  listening port: 51820

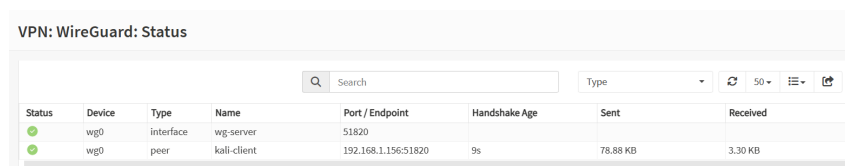
peer: Dh0mohaxz2lK7aUnNKd1zrgJjpQHplL10cDXS8t8KCwo=
  endpoint: 192.168.1.2:51820
  allowed ips: 10.100.0.0/24, 192.168.2.0/24
  latest handshake: 5 seconds ago
  transfer: 124 B received, 180 B sent
  persistent keepalive: every 25 seconds
```

Figure 3.43: WireGuard Connection Status on Kali (wg show)

The **sudo wg show** command confirms that the WireGuard interface **wg0** is active on Kali, connected to the OPNsense peer with a latest handshake 5 seconds ago and data being exchanged (124 B received, 180 B sent).

The allowed IPs **10.100.0.0/24** and **192.168.2.0/24** confirm that Kali can route traffic through the tunnel to the internal LAN (meaning any packet Kali sends to these address ranges will automatically go through the encrypted VPN tunnel instead of the regular internet connection).

The handshake was also confirmed from the **OPNsense side**. To allow VPN traffic to reach the internal LAN, the WireGuard interface was registered in OPNsense and a firewall rule was added to let that traffic through.



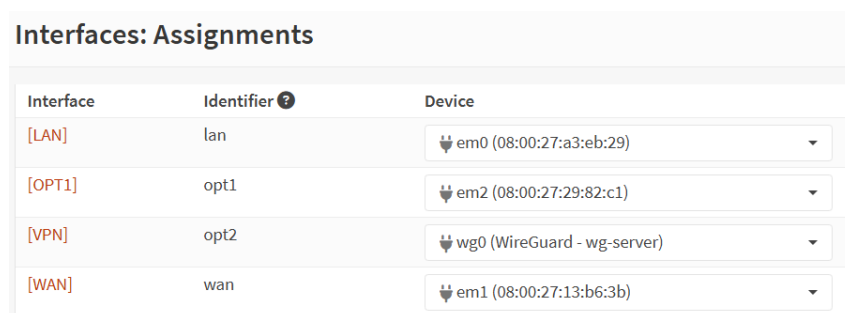
VPN: WireGuard: Status

Status	Device	Type	Name	Port / Endpoint	Handshake Age	Sent	Received
●	wg0	interface	wg-server	51820			
●	wg0	peer	kali-client	192.168.1.156:51820	9s	78.88 KB	3.30 KB

Figure 3.44: WireGuard Status on OPNsense

The OPNsense WireGuard status page confirms that both the **wg-server interface** and the **kali-client peer** are active with a green status. The handshake age of 9 seconds and data transfer of 78.88 KB sent / 3.30 KB received confirm that the tunnel is established and traffic is flowing.

To allow VPN traffic into the internal LAN, the WireGuard interface was added to OPNsense and a firewall rule was created to let it through.



Interfaces: Assignments

Interface	Identifier ?	Device
[LAN]	lan	em0 (08:00:27:a3:eb:29)
[OPT1]	opt1	em2 (08:00:27:29:82:c1)
[VPN]	opt2	wg0 (WireGuard - wg-server)
[WAN]	wan	em1 (08:00:27:13:b6:3b)

Figure 3.45: WireGuard Interface Assigned as VPN (OPT2)

The WireGuard tunnel interface wg0 was assigned as a new interface named VPN (OPT2) in OPNsense, making it possible to apply dedicated firewall rules to VPN traffic independently of other interfaces.

With the interface assigned, a dedicated VPN tab became available in the firewall rules section.

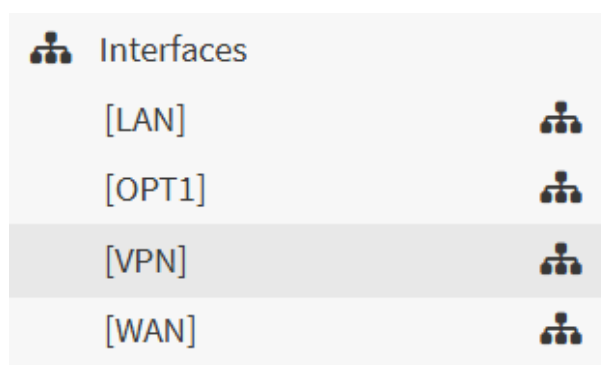


Figure 3.46: Dedicated VPN Interface Tab in Firewall Rules

A separate VPN tab is now visible in the OPNsense firewall rules alongside LAN, OPT1, and WAN, allowing specific rules to be applied exclusively to WireGuard tunnel traffic.

A firewall rule was then added on the VPN interface to allow traffic **from the tunnel to reach the LAN**.

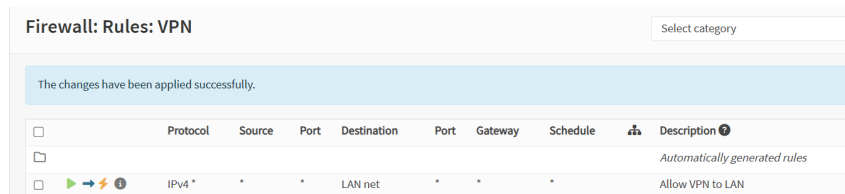


Figure 3.47: Firewall Rule: Allow VPN to LAN

A **Pass rule** was created on the VPN interface, allowing all IPv4 traffic from the WireGuard tunnel to reach the LAN net on any port.

This rule is what enables Kali, once connected to the VPN, to reach the internal LAN subnet 192.168.2.0/24 and access the OPNsense management GUI.

2.6 Sprint 2 Validation and Testing

To validate the complete VPN setup, two scenarios were tested: what happens when Kali is connected to the VPN, and what happens when the VPN is disconnected.

- **When VPN is Connected:**

```
(marjem@kali)-[~]
$ ping 192.168.2.1
PING 192.168.2.1 (192.168.2.1) 56(84) bytes of data:
64 bytes from 192.168.2.1: icmp_seq=1 ttl=64 time=3.79 ms
64 bytes from 192.168.2.1: icmp_seq=2 ttl=64 time=13.6 ms
64 bytes from 192.168.2.1: icmp_seq=3 ttl=64 time=7.50 ms
64 bytes from 192.168.2.1: icmp_seq=4 ttl=64 time=5.61 ms
64 bytes from 192.168.2.1: icmp_seq=5 ttl=64 time=6.09 ms
64 bytes from 192.168.2.1: icmp_seq=6 ttl=64 time=4.36 ms
64 bytes from 192.168.2.1: icmp_seq=7 ttl=64 time=6.28 ms
^C
— 192.168.2.1 ping statistics —
7 packets transmitted, 7 received, 0% packet loss, time 6484ms
rtt min/avg/max/mdev = 3.788/6.752/13.643/3.035 ms
```

Figure 3.48: Ping to OPNsense LAN Interface over VPN

A ping test to 192.168.2.1 confirms that Kali can reliably reach the OPNsense LAN interface through the encrypted tunnel, with 7/7 packets received and **0% packet loss**, confirming that the VPN tunnel is active and routing traffic correctly to the internal LAN.

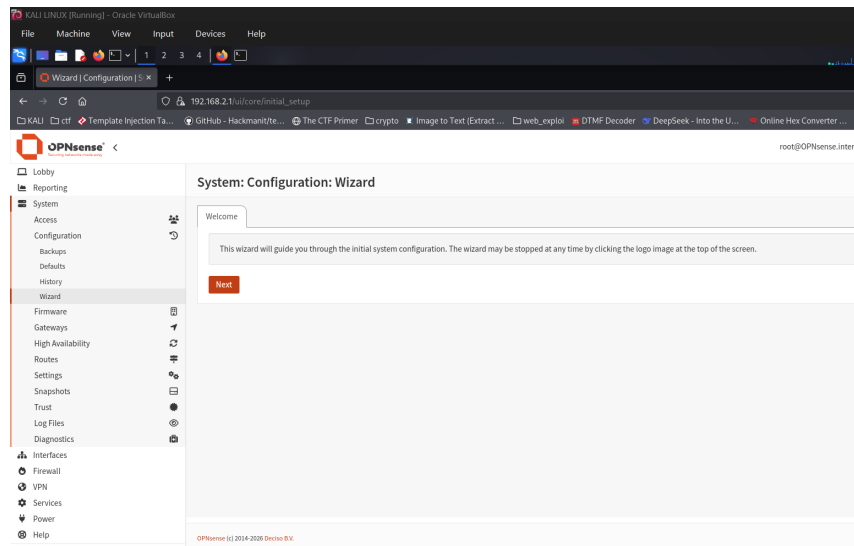


Figure 3.49: OPNsense Web GUI Accessed from Kali over VPN

Kali's browser successfully loaded the OPNsense management interface at **https://192.168.2.1**, proving that full remote administrative access to the internal network is working correctly through the VPN tunnel.



Figure 3.50: OPNsense GUI Source Code Retrieved via curl

The command **curl -k https://192.168.2.1** returned the full HTML source of the OPNsense login page, providing further confirmation that the LAN management interface is reachable from Kali over the VPN.

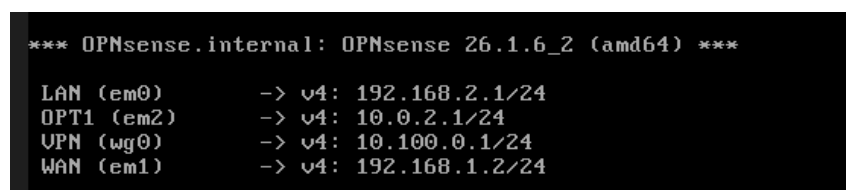
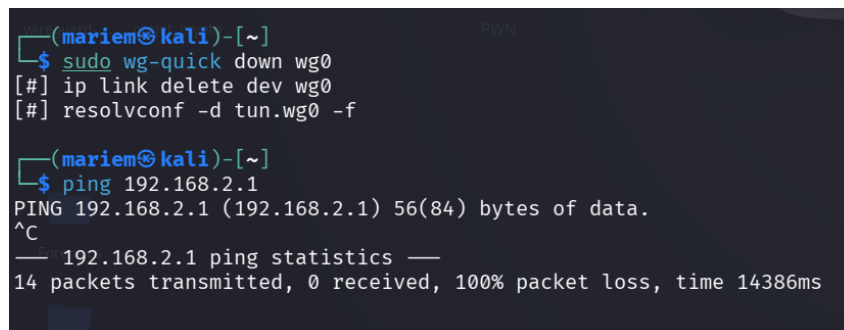


Figure 3.51: OPNsense Console: All Interfaces Active

The OPNsense console confirms all four interfaces are correctly active: **LAN**, **OPT1**, **VPN (wg0)**, and **WAN**, validating the complete network architecture.

- **When VPN is Disconnected :**



```
(mariem@kali)-[~]
$ sudo wg-quick down wg0
[#] ip link delete dev wg0
[#] resolvconf -d tun.wg0 -f

(mariem@kali)-[~]
$ ping 192.168.2.1
PING 192.168.2.1 (192.168.2.1) 56(84) bytes of data.
^C
--- 192.168.2.1 ping statistics ---
14 packets transmitted, 0 received, 100% packet loss, time 14386ms
```

Figure 3.52: VPN Disconnected : Ping to LAN Fails

After running **sudo wg-quick down wg0** (the command that shuts down the VPN tunnel), a ping to 192.168.2.1 resulted in 100% packet loss, confirming that the LAN is completely unreachable without the VPN.

2.7 Bonus : VPN Monitor Script

The goal of this bonus was to **build a script that alerts the administrator** whenever a peer's (the other device connected through the VPN, in this case Kali) WireGuard handshake goes silent, which can indicate a connectivity issue or an unauthorized access attempt. However, WireGuard works differently from other VPN protocols and this required a different approach.

- **Why Handshake Monitoring Instead of Log Parsing:**

WireGuard follows a "**silent rejection**" principle : when someone tries to connect using a wrong key, WireGuard simply ignores the packet and does not generate any log entry (a log entry is a recorded message that saves what happened and when, without it, there is no trace of the failed attempt anywhere).

This is intentional : by staying completely silent on failed attempts, WireGuard gives attackers no response and no log to read, making it impossible for them to even confirm that a VPN server is running.

As a result, the usual approach of reading logs to detect failed logins simply does not work here, because WireGuard never writes anything when a connection fails.

To overcome this limitation, the monitoring solution focuses on **WireGuard handshakes** instead of log files : a successful authentication always produces a handshake, so the absence of a recent handshake is useful signal to investigate, though on its own it does not confirm the cause (it could be a failed authentication attempt, or simply the peer being offline, disconnected or inactive).

The script checks the every 10 seconds using the **'wg show wg0 latest-handshakes'** command. When no handshake has been recorded for more than 180 seconds, a red warning message is displayed in the terminal so that the administrator can quickly detect and investigate the problem.

To simulate a peer becoming unreachable (as would happen with a key mismatch, for example), the Kali private key in the configuration file was on purpose replaced with an incorrect key .



```

GNU nano 8.7 /etc/wireguard/wg0.conf
[Interface]
PrivateKey = YGbGGbq3Gy8bSMDgKnFnKpgFwNzLmX7vR5cJqT2hU0A=
Address = 10.100.0.2/24
ListenPort = 51820
DNS = 192.168.2.1

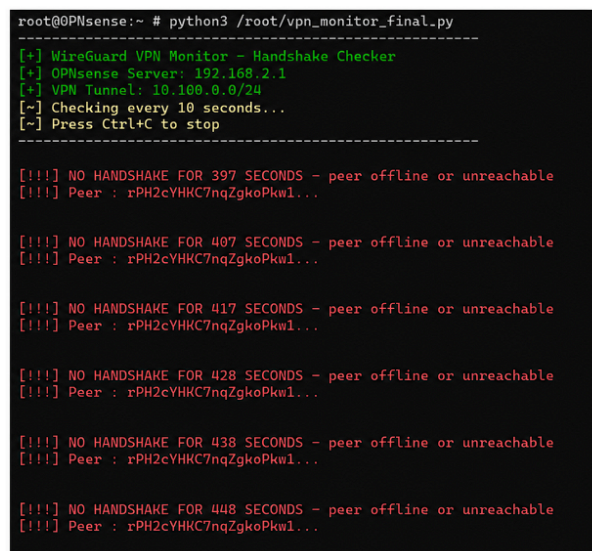
[Peer]
PublicKey = DhOmohaxz2lK7aUnNKd1zrgJjpQHpl10cDXS8t8KCwo=
Endpoint = 192.168.1.2:51820
AllowedIPs = 10.100.0.0/24, 192.168.2.0/24
PersistentKeepalive = 25

```

Figure 3.53: Kali WireGuard Configuration with Modified Private Key

The **wg0.conf** file on Kali was edited to replace the correct private key with a fake one, **simulating an unauthorized connection attempt**. The VPN was then restarted to trigger the failure scenario.

With the wrong key in place, the script immediately began detecting the missing handshake and alerting the administrator.



```

root@OPNsense:~ # python3 /root/vpn_monitor_final.py
-----
[+] WireGuard VPN Monitor - Handshake Checker
[+] OPNsense Server: 192.168.2.1
[+] VPN Tunnel: 10.100.0.0/24
[~] Checking every 10 seconds...
[~] Press Ctrl+C to stop
-----

[!!!] NO HANDSHAKE FOR 397 SECONDS - peer offline or unreachable
[!!!] Peer : rPH2cYHKC7nqZgkoPkw1...

[!!!] NO HANDSHAKE FOR 407 SECONDS - peer offline or unreachable
[!!!] Peer : rPH2cYHKC7nqZgkoPkw1...

[!!!] NO HANDSHAKE FOR 417 SECONDS - peer offline or unreachable
[!!!] Peer : rPH2cYHKC7nqZgkoPkw1...

[!!!] NO HANDSHAKE FOR 428 SECONDS - peer offline or unreachable
[!!!] Peer : rPH2cYHKC7nqZgkoPkw1...

[!!!] NO HANDSHAKE FOR 438 SECONDS - peer offline or unreachable
[!!!] Peer : rPH2cYHKC7nqZgkoPkw1...

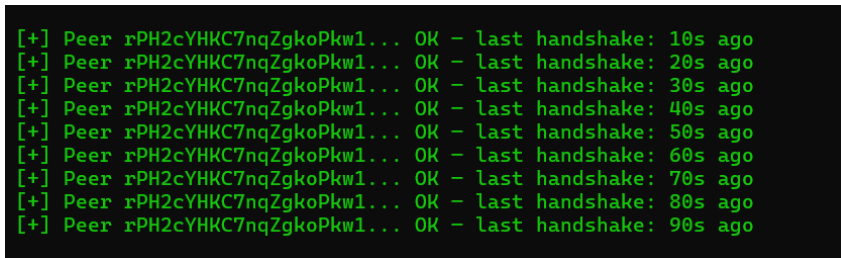
[!!!] NO HANDSHAKE FOR 448 SECONDS - peer offline or unreachable
[!!!] Peer : rPH2cYHKC7nqZgkoPkw1...

```

Figure 3.54: VPN Monitor Script: Red Alert :No Handshake Detected

The monitor script running on OPNsense detected that the Kali peer had not produced a handshake, triggering repeated **red terminal warnings** every 10 seconds. Each alert displays the peer's public key and the number of seconds since the last successful handshake, confirming that **the script correctly flags peer inactivity for the administrator to investigate**. (the script notices when a peer stops responding and warns the admin , without assuming the cause.)

The correct key was then restored to verify that the script returns to a normal state once the legitimate peer reconnects.



```
[+] Peer rPH2cYHKC7nqZgkoPkw1... OK - last handshake: 10s ago
[+] Peer rPH2cYHKC7nqZgkoPkw1... OK - last handshake: 20s ago
[+] Peer rPH2cYHKC7nqZgkoPkw1... OK - last handshake: 30s ago
[+] Peer rPH2cYHKC7nqZgkoPkw1... OK - last handshake: 40s ago
[+] Peer rPH2cYHKC7nqZgkoPkw1... OK - last handshake: 50s ago
[+] Peer rPH2cYHKC7nqZgkoPkw1... OK - last handshake: 60s ago
[+] Peer rPH2cYHKC7nqZgkoPkw1... OK - last handshake: 70s ago
[+] Peer rPH2cYHKC7nqZgkoPkw1... OK - last handshake: 80s ago
[+] Peer rPH2cYHKC7nqZgkoPkw1... OK - last handshake: 90s ago
```

Figure 3.55: VPN Monitor Script: Green Status : Handshake Restored

Once the private key was corrected and Kali connected to the Virtual Private Network again, the script immediately got back its **green ‘OK’ status**, proving that handshakes were occurring every 10 seconds. This proves that the monitoring script reliably distinguishes between an **active, healthy peer** and **one that has gone silent**, whatever the underlying cause.

2.8 Conclusion

Sprint 2 successfully established a secure encrypted remote access solution using **WireGuard VPN** on **OPNsense**. A full VPN tunnel was configured between **Kali** (WAN side) and the **internal LAN**, allowing secure administrative access to the **OPNsense management GUI**, a connection that was completely blocked without the VPN.

The checkpoint confirmed that **VPN access is the only authorized path** to the internal network.

As a bonus, a **Python monitoring script** was developed to monitor peer connectivity health through handshake tracking, simulating a basic health-monitoring layer for the VPN tunnel.

Chapter 4

PHASE II : Traffic Analysis & Active Defense

In this phase, we will generate real malicious traffic and learn how to identify and block it at the packet level. **Sprint 3** covers deep packet inspection and PCAP analysis, while **Sprint 4** introduces an IDS/IPS system using Suricata to automatically detect and block attacks in real time.

1 Sprint 3 : Deep Packet Inspection (DPI) & PCAP Analysis

The goal of this sprint is to simulate real attacks **from Kali** (WAN side) **against the Ubuntu web server** in the DMZ, capture the generated traffic and analyze it at the packet level using Wireshark.

1.1 Attack Simulation Overview

To generate realistic malicious traffic for analysis, three attacks were launched from the Kali VM (WAN side) against the Ubuntu web server in the DMZ. Each attack uses a dedicated tool and targets a different vulnerability:

- **Nmap** performs a port scan to discover which ports and services are open on the target.
- **hping3** launches a SYN flood attack, sending a massive number of fake connection requests to overwhelm the server.
- **Hydra** performs a brute-force attack, trying many passwords to break into the web server.

All three attacks target **the OPNsense WAN IP 192.168.1.2** rather than the Ubuntu server directly.

Ubuntu is isolated in the DMZ and not reachable from the WAN , traffic arriving at **192.168.1.2:80** is automatically redirected to **Ubuntu (10.0.2.10:80)** via **port forwarding**, making the firewall's WAN IP the only visible entry point from the outside, exactly as in a real-world scenario.

Before launching any attack, the **Python HTTP server** was started on Ubuntu to simulate an active public web server in the DMZ and make it a valid target for incoming malicious traffic.

1.1.1 Nmap Port Scan

Nmap is the industry-standard reconnaissance tool used by attackers to discover open ports and running services on a target. It was chosen as the first attack because **it generates a clear and recognizable traffic signature**, making it ideal for learning how to detect reconnaissance activity at the packet level.

Before launching any attack, an attacker needs to know what ports are open and what services are running, Nmap answers both in a single command.

The following command was executed from Kali:

```
└─$ sudo nmap -A -T4 192.168.1.2
[sudo] password for mariem:
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-08 12:58 +0000
Nmap scan report for 192.168.1.2
Host is up (0.0063s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
80/tcp    open  http      SimpleHTTPServer 0.6 (Python 3.12.3)
|_ http-server-header: SimpleHTTP/0.6 Python/3.12.3
|_ http-title: Directory listing for /
MAC Address: 08:00:27:13:B6:3B (Oracle VirtualBox virtual NIC)
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Device type: general purpose
Running (JUST GUESSING): Linux 4.X|5.X|2.6.X|3.X (92%)
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5 cpe:/o:linux:linux_kernel:2.6.32 cpe:/o:linux:linux_kernel:3.10
Aggressive OS guesses: Linux 4.19 - 5.15 (92%), Linux 4.15 - 5.19 (85%), Linux 5.0 - 5.14 (85%), OpenWrt 21.02 (Linux 5.4) (85%), Linux 2.6.32 (85%), Linux 2.6.32 or 3.10 (85%), Linux 4.0 - 4.4 (85%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop

TRACEROUTE
HOP RTT     ADDRESS
1   6.27 ms  192.168.1.2

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/.
Nmap done: 1 IP address (1 host up) scanned in 26.12 seconds
```

Figure 4.1: Nmap Scan Result: Reconnaissance Successful

The Nmap scan successfully fingerprinted the target: **port 80** was found **open** running the Python HTTP server, while **999 ports** were filtered by OPNsense : confirming that the firewall is correctly blocking all unnecessary traffic. Nmap also guessed the operating system as Linux 4.x/5.x (92%) just from packet behavior, proving how much information a simple scan can reveal to an attacker.

To confirm the attack was received on the target side, the Ubuntu server logs were checked.

Figure 4.2: Ubuntu Server Log: Nmap Scan Detected

The **200 OK responses** confirm that port 80 is reachable and the traffic was correctly delivered to the DMZ through port forwarding.

A SYN flood attack was chosen to simulate a real **Denial of Service (DoS)** scenario: it sends a **massive number of SYN packets without completing the TCP handshake**, exhausting the server's resources until it becomes unavailable. It was also selected because it tests the firewall and server under high-pressure traffic and produces a clear signature in Wireshark, which will be analyzed in the next section. The following command was executed from Kali:

Figure 4.3: SYN Flood Attack Completed: 14 Million Packets Sent

The hping3 output confirms that the packets were transmitted in flood mode with **0 packets received**, confirming that the server received no replies, as expected in a SYN

flood. This massive volume of fake connection requests was directed at port 80, simulating a realistic DoS attack against the web server in the DMZ.

The **100% packet loss** shown in the output is expected : This is because when hping3 is in flood mode it does not pay attention to the replies that come back. It does this on purpose so that it can keep sending packets fast, as possible. So the attack was working like it was supposed to work with hping3.

```
192.168.1.130 [100/30h/2020-12-31/20] GET / HTTP/1.1 200
[ 4881.552971] watchdog: BUG: soft lockup - CPU#0 stuck for 1923s! [swapper/0:0]
[ 6167.396079] watchdog: BUG: soft lockup - CPU#0 stuck for 993s! [swapper/0:0]
```

Figure 4.4: Ubuntu Server : CPU Soft Lockup During SYN Flood

The Ubuntu server log confirms the real impact of the attack : the kernel reported a **CPU soft lockup** on CPU0, meaning the processor was **so overwhelmed by the flood of incoming SYN packets** that it stopped responding normally. This is a direct consequence of the SYN flood and confirms that the attack successfully exhausted the server's resources.

1.1.3 Hydra : Brute Force

Hydra was chosen to replicate one of the most common attack techniques: **brute force password guessing**. Attackers frequently gain unauthorized access by automatically **trying thousands of commonly used passwords**.

Hydra automates this process by launching rapid brute-force login attempts against the web server, generating a high volume of repeated HTTP requests that produce a clear and identifiable pattern at the packet level.

The following command was executed from Kali:

```
(kali@kali:~)$ hydra -l admin -P /usr/share/wordlists/rockyou.txt 192.168.1.2 http-get /
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-06-08 15:01:16
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (l1/p:14344399), ~896525 tries per task
[DATA] attacking http-get://192.168.1.2:80/
[80] http-get host: 192.168.1.2 login: admin password: password
[80] http-get host: 192.168.1.2 login: admin password: iloveyou
[80] http-get host: 192.168.1.2 login: admin password: princess
[80] http-get host: 192.168.1.2 login: admin password: 1234567
[80] http-get host: 192.168.1.2 login: admin password: 123456789
[80] http-get host: 192.168.1.2 login: admin password: rockyou
[80] http-get host: 192.168.1.2 login: admin password: 12345678
[80] http-get host: 192.168.1.2 login: admin password: abc123
[80] http-get host: 192.168.1.2 login: admin password: nicole
[80] http-get host: 192.168.1.2 login: admin password: babygirl
[80] http-get host: 192.168.1.2 login: admin password: daniel
[80] http-get host: 192.168.1.2 login: admin password: lovely
[80] http-get host: 192.168.1.2 login: admin password: money
[80] http-get host: 192.168.1.2 login: admin password: 12345
[80] http-get host: 192.168.1.2 login: admin password: 123456
[80] http-get host: 192.168.1.2 login: admin password: jessica
1 of 1 target successfully completed, 16 valid passwords found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-06-08 15:01:18
```

Figure 4.5: Hydra Brute-Force Attack Running on Kali

The **-l admin** sets the username and **-P** points to the **rockyou.txt** wordlist containing millions of real-world passwords.

Hydra loops through every password, sending an **HTTP GET** request for each attempt against **192.168.1.2**, which forwards the traffic to the Ubuntu server in the DMZ.

During the initial test, Hydra generated a large number of HTTP requests containing different credential combinations. The apparent successful results reported by Hydra were caused by the Python HTTP server returning successful responses without validating the credentials.

After enabling IPS mode, Hydra encountered repeated connection errors and could no longer complete the same sequence of credential-bearing HTTP requests. This demonstrates that the matching traffic was interrupted under the configured IPS rules; it does not indicate that valid passwords had previously been discovered.

The attack was also confirmed on the target side through the Ubuntu server logs.

```
[ 6167.396079] watchdog: BUG: soft lockup - CPU#0 stuck for 993s! [swa]
192.168.1.156 - - [08/Jun/2026 14:39:35] "GET / HTTP/1.1" 200 -
192.168.1.156 - - [08/Jun/2026 14:39:35] "GET / HTTP/1.1" 200 -
192.168.1.156 - - [08/Jun/2026 14:39:35] "GET / HTTP/1.1" 200 -
192.168.1.156 - - [08/Jun/2026 14:39:35] "GET / HTTP/1.1" 200 -
192.168.1.156 - - [08/Jun/2026 14:39:35] "GET / HTTP/1.1" 200 -
192.168.1.156 - - [08/Jun/2026 14:39:35] "GET / HTTP/1.1" 200 -
192.168.1.156 - - [08/Jun/2026 14:39:35] "GET / HTTP/1.1" 200 -
192.168.1.156 - - [08/Jun/2026 14:39:35] "GET / HTTP/1.1" 200 -
192.168.1.156 - - [08/Jun/2026 14:39:35] "GET / HTTP/1.1" 200 -
192.168.1.156 - - [08/Jun/2026 14:39:35] "GET / HTTP/1.1" 200 -
192.168.1.156 - - [08/Jun/2026 14:39:35] "GET / HTTP/1.1" 200 -
192.168.1.156 - - [08/Jun/2026 14:39:35] "GET / HTTP/1.1" 200 -
192.168.1.156 - - [08/Jun/2026 14:39:35] "GET / HTTP/1.1" 200 -
192.168.1.156 - - [08/Jun/2026 14:39:35] "GET / HTTP/1.1" 200 -
192.168.1.156 - - [08/Jun/2026 14:39:35] "GET / HTTP/1.1" 200 -
192.168.1.156 - - [08/Jun/2026 14:39:35] "GET / HTTP/1.1" 200 -
192.168.1.156 - - [08/Jun/2026 14:39:35] "GET / HTTP/1.1" 200 -
192.168.1.156 - - [08/Jun/2026 14:39:35] "GET / HTTP/1.1" 200 -
```

Figure 4.6: Ubuntu Server: Brute-Force Traffic Received

The Ubuntu server log confirms a flood of **HTTP GET requests** from Kali (**192.168.1.156**), each answered with a **200 OK response**, confirming that the brute-force traffic was successfully delivered to the DMZ through port forwarding.

1.2 Packet Capture

With all three attacks launched, the generated traffic was captured directly on the OPNsense WAN interface for later analysis in **Wireshark**.

1.2.1 Start Capture on OPNsense WAN Interface

Interfaces: Diagnostics: Packet Capture

Capture

Jobs

1 Interface	em1 [WAN]	☒
1 Promiscuous	☒	
1 Address Family	IPv4 Only	
1 Invert Protocol	☐	
1 Protocol	any	
1 Host Address		
1 Invert Port	☐	
1 Port		
1 Packet Length		
1 Count	0	
1 Description	Attacks from Kali	

Start

Figure 4.7: Start Capture on OPNsense WAN Interface

The packet capture was configured on the WAN interface (em1) in **promiscuous mode**, capturing all incoming IPv4 traffic.

The capture was labeled "Attacks from Kali" to record all malicious traffic from the three attacks into a **.pcap** file for Wireshark analysis.

Interfaces: Diagnostics: Packet Capture

Capture
Jobs

Description
Attacks from Kali

«
<
i
>
»

Figure 4.8: Packet Capture Job Running

The capture job "Attacks from Kali" is active and recording all incoming attack traffic from Kali on the WAN interface.

1.2.2 Download the .pcap File

Figure 4.9: Downloading the .pcap File from OPNsense

Once all attacks were completed, the capture was stopped and the **.pcap** file was downloaded from the OPNsense packet capture interface.

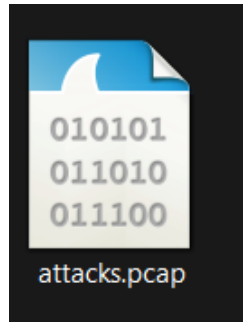


Figure 4.10: attacks.pcap File

The downloaded file **attacks.pcap** contains all recorded malicious traffic from the Nmap scan, SYN flood, and Hydra brute-force attacks, ready to be opened and analyzed in Wireshark.

1.3 Wireshark Analysis

The **attacks.pcap** file was opened in Wireshark on the host PC to analyze each attack signature at the packet level. By identifying the specific patterns generated by each attack, we can later write precise IDS/IPS rules to detect and block them automatically in Sprint 4.

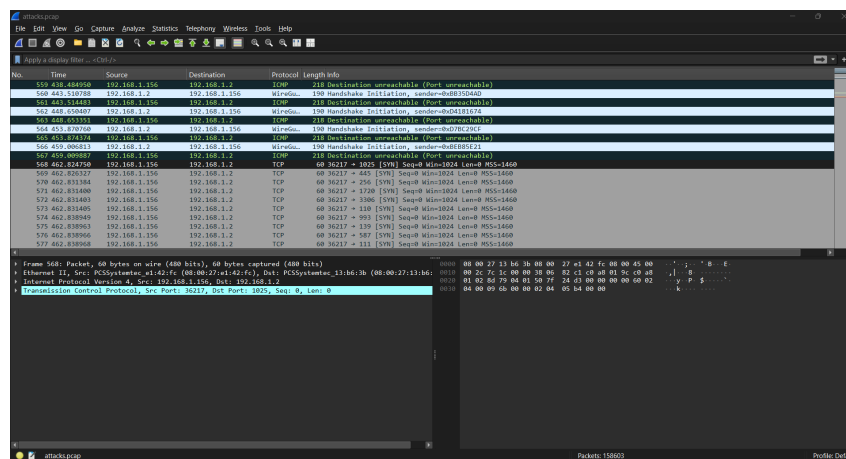


Figure 4.11: Wireshark Overview: attacks.pcap

The capture file contains mixed traffic including ICMP, WireGuard, and TCP packets from all three attacks. Specific filters will now be applied to isolate and identify each attack signature individually.

1.3.1 Nmap Scan Analysis

The filter **tcp.flags.syn==1** was applied to isolate SYN packets and identify the Nmap scan signature.

No.	Time	Source	Destination	Protocol	Length	Info
2951	483.408178	192.168.1.2	192.168.1.156	TCP	74	80 → 59242 [SYN, ACK] Seq=0 Ack=1 Win=65536 Len=0 MSS=1460 SACK_PERM TSval=2777868356 TSecr=2866714864 WS=128
2953	483.410129	192.168.1.2	192.168.1.156	TCP	74	80 → 59250 [SYN, ACK] Seq=0 Ack=1 Win=65536 Len=0 MSS=1460 SACK_PERM TSval=2777868358 TSecr=2866714864 WS=128
2970	483.465538	192.168.1.156	192.168.1.2	TCP	74	59262 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2866714921 TSecr=0 WS=512
2977	483.475763	192.168.1.2	192.168.1.156	TCP	74	80 → 59262 [SYN, ACK] Seq=0 Ack=1 Win=65536 Len=0 MSS=1460 SACK_PERM TSval=2777868358 TSecr=2866714921 WS=128
2986	483.519739	192.168.1.156	192.168.1.2	TCP	74	59278 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2866714980 TSecr=0 WS=512
2989	483.520134	192.168.1.156	192.168.1.2	TCP	74	59282 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2866714983 TSecr=0 WS=512
2992	483.524893	192.168.1.2	192.168.1.156	TCP	74	80 → 59278 [SYN, ACK] Seq=0 Ack=1 Win=65536 Len=0 MSS=1460 SACK_PERM TSval=2777868472 TSecr=2866714980 WS=128
2993	483.527527	192.168.1.2	192.168.1.156	TCP	74	80 → 59282 [SYN, ACK] Seq=0 Ack=1 Win=65536 Len=0 MSS=1460 SACK_PERM TSval=2777868473 TSecr=2866714983 WS=128
3000	483.578791	192.168.1.156	192.168.1.2	TCP	74	59296 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2866715042 TSecr=0 WS=512
3004	483.583961	192.168.1.2	192.168.1.156	TCP	74	80 → 59296 [SYN, ACK] Seq=0 Ack=1 Win=65536 Len=0 MSS=1460 SACK_PERM TSval=2777868533 TSecr=2866715042 WS=128
3026	483.701600	192.168.1.156	192.168.1.2	TCP	74	59306 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2866715165 TSecr=0 WS=512
3028	483.704421	192.168.1.2	192.168.1.156	TCP	74	80 → 59306 [SYN, ACK] Seq=0 Ack=1 Win=65536 Len=0 MSS=1460 SACK_PERM TSval=2777868654 TSecr=2866715165 WS=128
3032	483.763786	192.168.1.156	192.168.1.2	TCP	74	59308 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2866715227 TSecr=0 WS=512
3035	483.767629	192.168.1.2	192.168.1.156	TCP	74	80 → 59308 [SYN, ACK] Seq=0 Ack=1 Win=65536 Len=0 MSS=1460 SACK_PERM TSval=2777868717 TSecr=2866715227 WS=128
3049	483.881081	192.168.1.156	192.168.1.2	TCP	74	59314 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2866715344 TSecr=0 WS=512
3051	483.884534	192.168.1.2	192.168.1.156	TCP	74	80 → 59314 [SYN, ACK] Seq=0 Ack=1 Win=65536 Len=0 MSS=1460 SACK_PERM TSval=2777868834 TSecr=2866715344 WS=128
3063	487.580927	192.168.1.156	192.168.1.2	TCP	74	59326 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2866719027 TSecr=0 WS=512
3064	487.563761	192.168.1.2	192.168.1.156	TCP	74	80 → 59326 [SYN, ACK] Seq=0 Ack=1 Win=65536 Len=0 MSS=1460 SACK_PERM TSval=2777872514 TSecr=2866719027 WS=128
3072	487.581987	192.168.1.156	192.168.1.2	TCP	74	59340 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2866719048 TSecr=0 WS=512

Figure 4.12: Nmap Port Sweep Captured in Wireshark

The filter confirms that Kali (**192.168.1.156**) sent SYN packets to multiple ports on OPNsense (**192.168.1.2**). Only **port 80** replied with a **SYN-ACK** because it is the only port with an **active service**, the Python HTTP server is running on it and is reachable through the NAT port forwarding rule. All other ports gave no response since nothing is listening on them.

1.3.2 SYN Flood Analysis

The filter **tcp.flags.syn==1** and **tcp.flags.ack==0** and **ip.src==192.168.1.156** was applied to isolate **pure SYN** packets from Kali with **no ACK**, exactly the abnormal behavior produced by a SYN flood.

No.	Time	Source	Destination	Protocol	Length	Info
2707	482.337775	192.168.1.156	192.168.1.2	TCP	74	59030 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2866713801 TSecr=0 WS=512
2708	482.342019	192.168.1.156	192.168.1.2	TCP	74	59042 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2866713803 TSecr=0 WS=512
2709	482.346698	192.168.1.156	192.168.1.2	TCP	74	59058 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2866713804 TSecr=0 WS=512
2710	482.347372	192.168.1.156	192.168.1.2	TCP	74	59060 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2866713805 TSecr=0 WS=512
2713	482.349903	192.168.1.156	192.168.1.2	TCP	74	59068 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2866713807 TSecr=0 WS=512
2714	482.350523	192.168.1.156	192.168.1.2	TCP	74	59084 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2866713808 TSecr=0 WS=512
2715	482.350969	192.168.1.156	192.168.1.2	TCP	74	59096 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2866713809 TSecr=0 WS=512
2728	482.378933	192.168.1.156	192.168.1.2	TCP	74	59102 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2866713824 TSecr=0 WS=512
2729	482.371486	192.168.1.156	192.168.1.2	TCP	74	59104 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2866713826 TSecr=0 WS=512
2730	482.372737	192.168.1.156	192.168.1.2	TCP	74	59112 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2866713827 TSecr=0 WS=512
2731	482.373958	192.168.1.156	192.168.1.2	TCP	74	59122 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2866713828 TSecr=0 WS=512
2732	482.376337	192.168.1.156	192.168.1.2	TCP	74	59126 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2866713828 TSecr=0 WS=512
2799	482.596164	192.168.1.156	192.168.1.2	TCP	74	59136 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2866714059 TSecr=0 WS=512
2804	482.599982	192.168.1.156	192.168.1.2	TCP	74	59152 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2866714062 TSecr=0 WS=512
2824	482.684457	192.168.1.156	192.168.1.2	TCP	74	59158 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2866714144 TSecr=0 WS=512
2825	482.686691	192.168.1.156	192.168.1.2	TCP	74	59172 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2866714144 TSecr=0 WS=512
2856	482.807723	192.168.1.156	192.168.1.2	TCP	74	59180 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2866714269 TSecr=0 WS=512
2857	482.808822	192.168.1.156	192.168.1.2	TCP	74	59186 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2866714269 TSecr=0 WS=512
2865	482.878762	192.168.1.156	192.168.1.2	TCP	74	59196 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2866714340 TSecr=0 WS=512

Figure 4.13: SYN Flood: Thousands of SYN Packets Captured

Kali (**192.168.1.156**) is hammering **port 80** on OPNsense (**192.168.1.2**) with thousands of SYN packets in rapid succession, all with different source ports but never followed by an ACK. The server is forced to allocate resources waiting for connections that will never complete : this is precisely how a SYN flood exhausts a target.

```

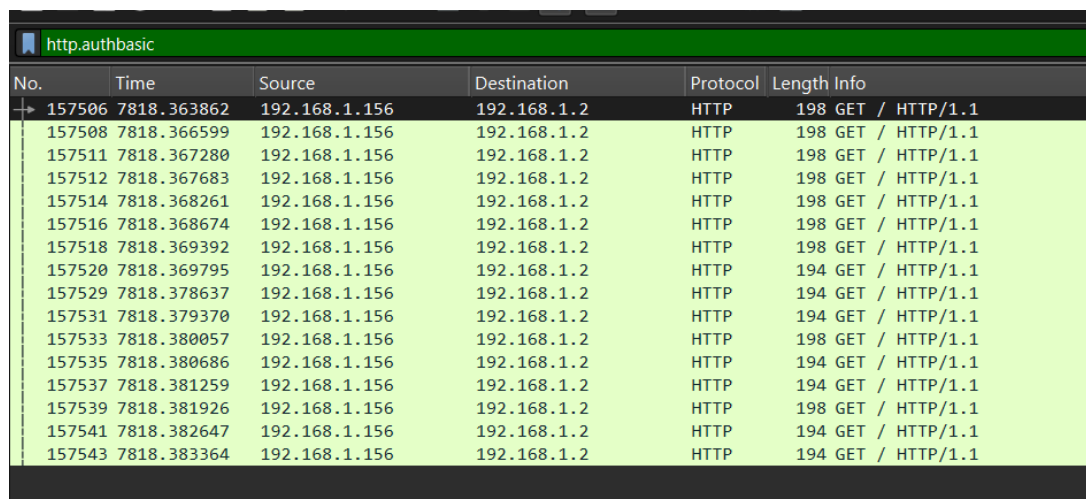
Transmission Control Protocol, Src Port: 36217, Dst Port: 1025, Seq: 0, Len: 0
  Source Port: 36217
  Destination Port: 1025
  [Stream index: 0]
  [Stream Packet Number: 1]
  [Conversation completeness: Incomplete, SYN_SENT (1)]
    ... .. = RST: Absent
    ... .. = FIN: Absent
    ... .. = Data: Absent
    ... .. = ACK: Absent
    ... .. = SYN-ACK: Absent
    ... .. = SYN: Present
  [Completeness Flags: .....S]
  [TCP Segment Len: 0]
  Sequence Number: 0 (relative sequence number)
  Sequence Number (raw): 1350509779
  [Next Sequence Number: 1 (relative sequence number)]
  Acknowledgment Number: 0
  Acknowledgment number (raw): 0
  0110 .... = Header Length: 24 bytes (6)
  
```

Figure 4.14: SYN Flood: Incomplete Handshake Signature

The packet detail confirms the SYN flood signature: **SYN is present** while **ACK and SYN-ACK are both absent**, and the conversation is marked as Incomplete, **SYN-SENT** : meaning Kali sent the connection request but never completed the handshake. Multiplied by millions of packets, this is what exhausts the server's resources.

1.3.3 Hydra Brute Force Analysis

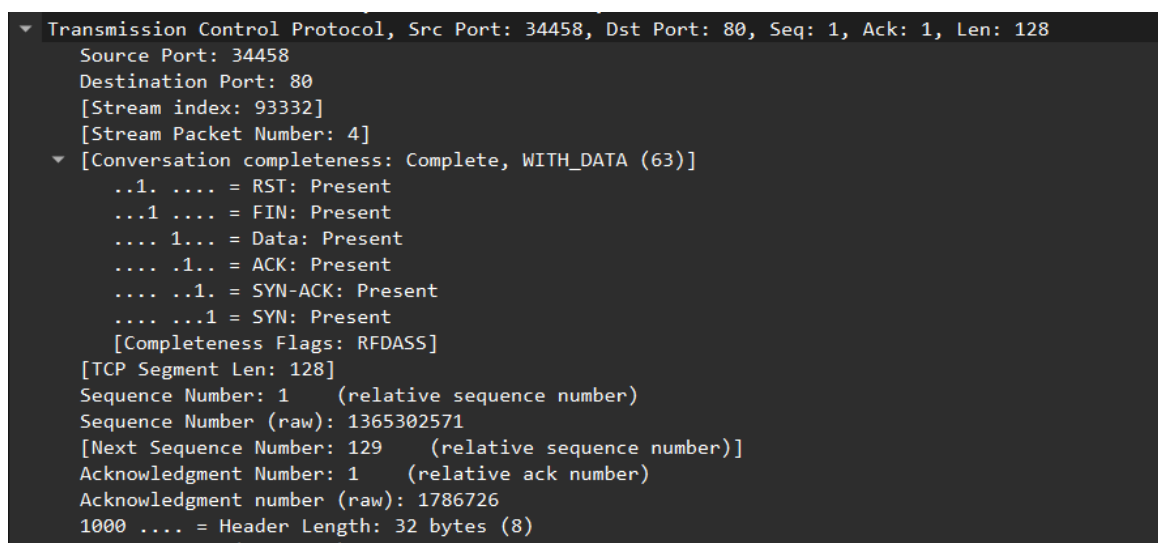
The filter **http.authbasic** was applied to isolate only HTTP packets containing a **Basic Authentication** header, showing exclusively the brute-force password attempts.



No.	Time	Source	Destination	Protocol	Length Info
157506	7818.363862	192.168.1.156	192.168.1.2	HTTP	198 GET / HTTP/1.1
157508	7818.366599	192.168.1.156	192.168.1.2	HTTP	198 GET / HTTP/1.1
157511	7818.367280	192.168.1.156	192.168.1.2	HTTP	198 GET / HTTP/1.1
157512	7818.367683	192.168.1.156	192.168.1.2	HTTP	198 GET / HTTP/1.1
157514	7818.368261	192.168.1.156	192.168.1.2	HTTP	198 GET / HTTP/1.1
157516	7818.368674	192.168.1.156	192.168.1.2	HTTP	198 GET / HTTP/1.1
157518	7818.369392	192.168.1.156	192.168.1.2	HTTP	198 GET / HTTP/1.1
157520	7818.369795	192.168.1.156	192.168.1.2	HTTP	194 GET / HTTP/1.1
157529	7818.378637	192.168.1.156	192.168.1.2	HTTP	194 GET / HTTP/1.1
157531	7818.379370	192.168.1.156	192.168.1.2	HTTP	194 GET / HTTP/1.1
157533	7818.380057	192.168.1.156	192.168.1.2	HTTP	198 GET / HTTP/1.1
157535	7818.380686	192.168.1.156	192.168.1.2	HTTP	194 GET / HTTP/1.1
157537	7818.381259	192.168.1.156	192.168.1.2	HTTP	194 GET / HTTP/1.1
157539	7818.381926	192.168.1.156	192.168.1.2	HTTP	198 GET / HTTP/1.1
157541	7818.382647	192.168.1.156	192.168.1.2	HTTP	194 GET / HTTP/1.1
157543	7818.383364	192.168.1.156	192.168.1.2	HTTP	194 GET / HTTP/1.1

Figure 4.15: Hydra Brute Force: HTTP Auth Attempts Captured

Every packet is a **GET / HTTP/1.1** from Kali (**192.168.1.156**) to OPNsense (**192.168.1.2**), each carrying a different password in the Authorization header. The machine-speed repetition with identical destination and request path is the unmistakable signature of Hydra's brute-force attack.



```

Transmission Control Protocol, Src Port: 34458, Dst Port: 80, Seq: 1, Ack: 1, Len: 128
  Source Port: 34458
  Destination Port: 80
  [Stream index: 93332]
  [Stream Packet Number: 4]
  [Conversation completeness: Complete, WITH_DATA (63)]
    ..1. .... = RST: Present
    ...1 .... = FIN: Present
    .... 1... = Data: Present
    .... .1.. = ACK: Present
    .... ..1. = SYN-ACK: Present
    .... ...1 = SYN: Present
  [Completeness Flags: RFDASS]
  [TCP Segment Len: 128]
  Sequence Number: 1 (relative sequence number)
  Sequence Number (raw): 1365302571
  [Next Sequence Number: 129 (relative sequence number)]
  Acknowledgment Number: 1 (relative ack number)
  Acknowledgment number (raw): 1786726
  1000 .... = Header Length: 32 bytes (8)
  
```

Figure 4.16: Hydra Brute Force: Complete Connection Per Attempt

Unlike the SYN flood which never completes the handshake, each Hydra attempt

establishes a **full HTTP connection** , **SYN**, **ACK**, **SYN-ACK**, and Data all present : sends the password, then immediately closes the connection with **RST** and **FIN**.

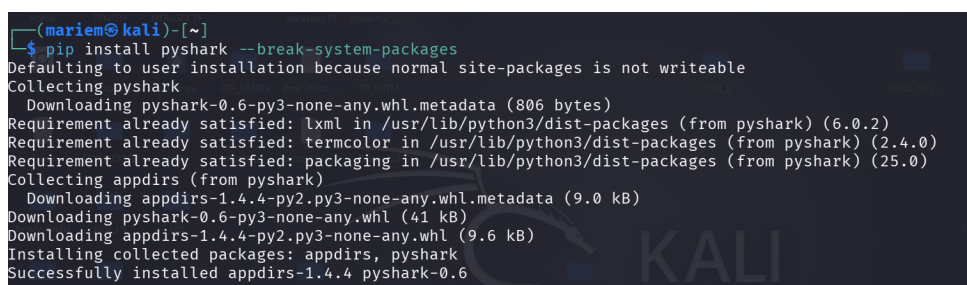
This confirms that Hydra completes thousands of real connections per second, one per password attempt.

1.4 Bonus : Programmatic PCAP Parsing

Instead of manually reading packets in Wireshark, a Python script was written to **automatically analyze the .pcap file** and identify the **top 5 most active IP addresses**.

In real cybersecurity, analysts use scripts like this to quickly spot attackers in large capture files where manual inspection is impossible.

1.4.1 Installing pyshark



```
(mariem@kali)-[~]
$ pip install pyshark --break-system-packages
Defaulting to user installation because normal site-packages is not writeable
Collecting pyshark
  Downloading pyshark-0.6-py3-none-any.whl.metadata (806 bytes)
Requirement already satisfied: lxml in /usr/lib/python3/dist-packages (from pyshark) (6.0.2)
Requirement already satisfied: termcolor in /usr/lib/python3/dist-packages (from pyshark) (2.4.0)
Requirement already satisfied: packaging in /usr/lib/python3/dist-packages (from pyshark) (25.0)
Collecting appdirs (from pyshark)
  Downloading appdirs-1.4.4-py2.py3-none-any.whl.metadata (9.0 kB)
Downloading pyshark-0.6-py3-none-any.whl (41 kB)
Downloading appdirs-1.4.4-py2.py3-none-any.whl (9.6 kB)
Installing collected packages: appdirs, pyshark
Successfully installed appdirs-1.4.4 pyshark-0.6
```

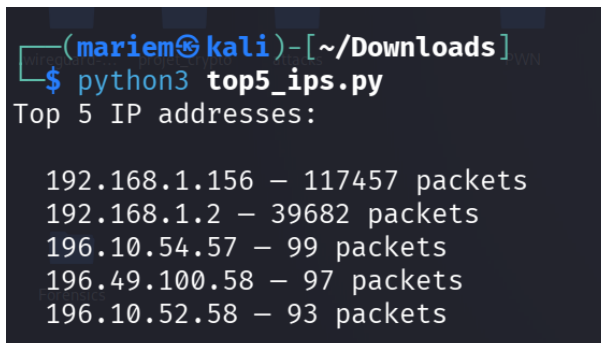
Figure 4.17: pyshark Installation on Kali

pyshark is a Python wrapper for **tshark** : Wireshark's command-line engine. Installing it allows Python scripts to **read and analyze .pcap files programmatically**, automating the analysis instead of clicking through Wireshark manually.

1.4.2 Python Script : Top 5 IP Addresses

By extracting the top 5 IPs generating the most traffic, the script instantly identifies the most active sources **without reading packet by packet**.

In a real production network with hundreds of unknown IPs, this script would immediately highlight suspicious sources.



```
(mariem@kali)-[~/Downloads]
$ python3 top5_ips.py
Top 5 IP addresses:

192.168.1.156 - 117457 packets
192.168.1.2 - 39682 packets
196.10.54.57 - 99 packets
196.49.100.58 - 97 packets
196.10.52.58 - 93 packets
```

Figure 4.18: pyshark Script Output: Top 5 IP Addresses

The script uses **pyshark** to open the attacks.pcap file and **Counter** to count packet occurrences by source IP. It loops through every packet, reads the source IP, and skips

non-IP packets automatically. The top 5 IPs are then printed from most to least active, confirming the attack traffic:

- **192.168.1.156 : Kali** (the attacker), dominated by the SYN flood and Hydra brute-force attacks.
- **192.168.1.2 : OPNsense firewall**, responding to the attacks.

196.10.54.57, 196.49.100.58, 196.10.52.58 : background internet traffic, unrelated to the attacks.

1.5 Conclusion

Sprint 3 successfully simulated three real-world attacks : **Nmap port scan**, **SYN flood**, and **Hydra brute-force** , from Kali against the Ubuntu web server in the DMZ.

All attack traffic was captured **on the OPNsense WAN interface** and analyzed in **Wireshark**, where each attack's **signature** was clearly identified at the packet level.

As a bonus, a **Python script** using **pyshark** automatically identified the **top attacking IPs** from the capture file.

These findings now serve as the foundation for Sprint 4, where Suricata will be configured to automatically detect and block these attacks in real time.

2 Sprint 4 : Intrusion Detection System (IDS) & Prevention

In Sprint 3 you played the attacker : you generated malicious traffic and learned to identify it.

This sprint, we **switch sides**. we will deploy Suricata: a real IDS/IPS engine, directly on your OPNsense firewall to automatically detect and block those same attacks before they ever reach the DMZ.

This is where your lab becomes a **real security system**. Not just a network that gets attacked, but one that fights back automatically.

2.1 Suricata Deployment

2.1.1 Install Suricata on OPNsense

Suricata is a free, open-source IDS/IPS engine. It reads every packet passing through your firewall and compares it against rules , if a packet matches a known attack pattern, it either **alerts (IDS mode)** or **drops it (IPS mode)**.

In this lab, Suricata sits directly on OPNsense and watches everything hitting the WAN and DMZ interfaces.



Figure 4.19: Suricata logo

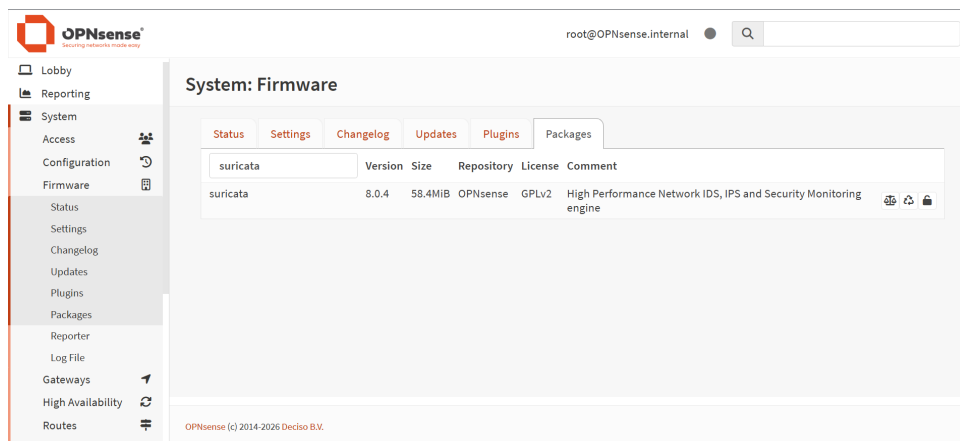


Figure 4.20: Suricata package installed in OPNsense Firmware

Suricata v8.0.4 is now installed directly on the firewall and ready to be configured.

2.1.2 Configure Suricata to Monitor the WAN Interface

WAN is where all attack traffic enters first : Nmap, SYN flood, and Hydra **all hit this interface before anything else**. Monitoring here means Suricata sees every attack **the moment it arrives**.

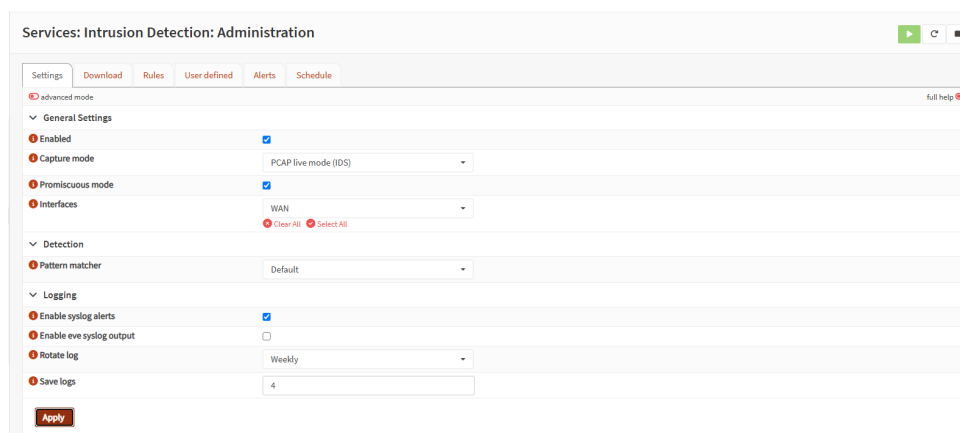


Figure 4.21: Suricata configured on WAN interface

Suricata is enabled and watching the WAN interface in **PCAP live mode (IDS)**, passive detection only, no packets are dropped yet. Promiscuous mode is checked so it reads every packet passing through, even those not addressed directly to the firewall. Syslog alerts is enabled, which will be used in the bonus step.

2.1.3 Add the DMZ Interface

WAN monitoring alone isn't enough. Your NAT rule forwards port 80 directly to the Ubuntu DMZ server, meaning Hydra and Nmap traffic actually **reaches the DMZ after passing through the firewall**.

Monitoring the DMZ interface lets Suricata confirm whether an attack was stopped at the WAN level or made it all the way to the target.

This is defense in depth : multiple detection layers so nothing slips through undetected.

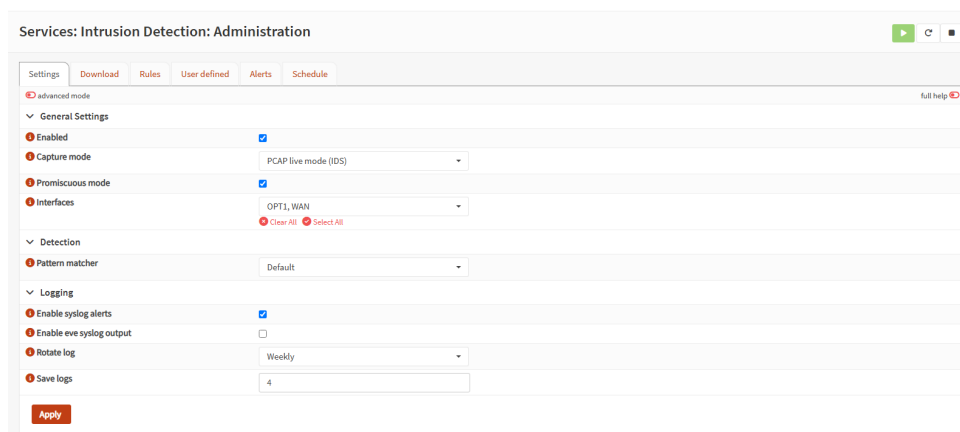


Figure 4.22: Suricata configured on OPT1 (DMZ) + WAN

Suricata now monitors both interfaces simultaneously. Every packet entering from Kali is caught at WAN, and any traffic reaching the Ubuntu server is caught at the DMZ level : **full visibility across the entire attack path.**

2.1.4 Download the Emerging Threats Community Ruleset

Suricata by itself has no rules , it doesn't know what an attack looks like until you give it signatures. **Emerging Threats** is a free, community-maintained ruleset covering thousands of known attack patterns. We enable **three rulesets** specifically, each one mapping directly to an attack simulated in Sprint 3.

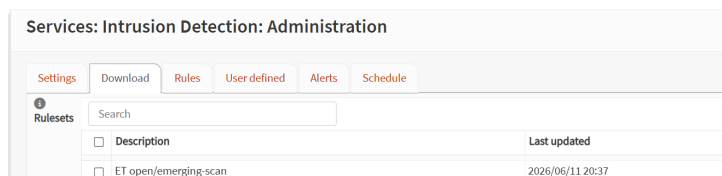


Figure 4.23: ET open/emerging-scan

The scan ruleset gives Suricata **the signatures** it needs to detect Nmap port scanning activity : the exact traffic pattern from Attack 1.

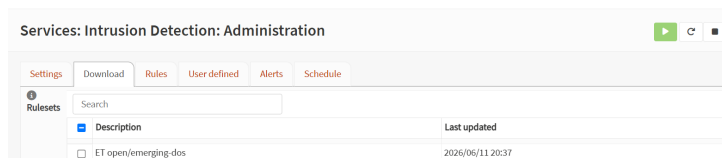
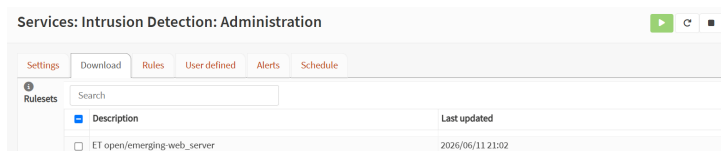


Figure 4.24: ET open/emerging-dos

The DoS ruleset covers SYN flood patterns : what hping3 generates. It teaches Suricata to recognize an **abnormal volume of SYN packets** and trigger an alert immediately.

Figure 4.25: ET open/emerging-web_server

The web server ruleset covers HTTP-layer attacks like Hydra brute-force on port 80 : rapid login attempts and suspicious request patterns targeting the Ubuntu server.

2.2 Custom Rule Creation

The Emerging Threats rulesets cover general known attacks, but they are not tailored to our specific lab environment. Custom rules let us write **precise signatures** targeting exactly what we observed in Sprint 3 : our specific IP addresses, our specific tools, our specific traffic patterns.

In a real SOC, analysts don't just rely on community rules , they write their own based on what they observe on their network. This is the direct bridge between Sprint 3 (identify the attack) and Sprint 4 (block it automatically).

2.2.1 Custom Rule: Nmap Port Scan Detection

Back in Sprint 3, Wireshark revealed Nmap's exact signature: rapid SYN packets with no ACK, all originating from Kali (192.168.1.156) targeting OPNsense (192.168.1.2). We now translate that observation into a Suricata rule , so instead of a human spotting it manually, Suricata detects it automatically the moment it happens.

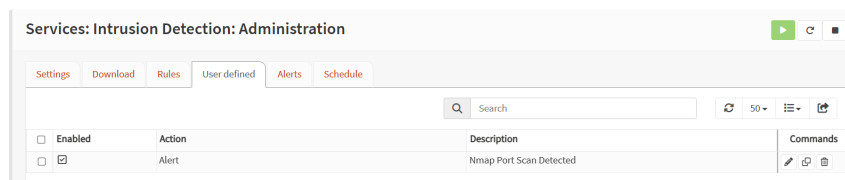


Figure 4.26: Custom rule: Nmap Port Scan Detected

The rule is set to Alert mode , every time Suricata sees Nmap's traffic pattern, it logs an alert without blocking yet. This is intentional: we first confirm the rule fires correctly before switching to drop mode in the IPS step.

2.2.2 Custom Rule: Hydra Brute-Force Detection

In Sprint 3, Wireshark showed Hydra sending hundreds of HTTP GET requests per second from Kali to port 80, each carrying a Basic Authentication header with a different password. We now translate that pattern into a rule.

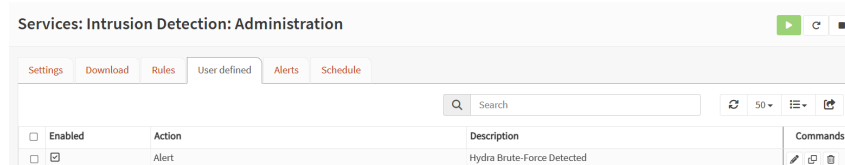


Figure 4.27: Custom rule: Hydra Brute-Force Detected

Like the Nmap rule, it is set to Alert mode for now . Suricata logs every brute-force attempt without blocking, allowing us to verify the rule works correctly before enabling full prevention mode.

2.2.3 Testing: Confirming Alerts Fire Before Switching to IPS

Before switching to blocking mode, we need to **confirm the custom rules actually work**. We re-launch attacks from Kali and watch the Suricata Alerts tab in real time.

```
(marion@kali)-[~]
$ sudo nmap -A -T4 192.168.1.2
[sudo] password for marion:
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-11 22:19 +0000
Nmap scan report for 192.168.1.2
Host is up (0.0036s latency).
All 1000 scanned ports on 192.168.1.2 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)
MAC Address: 08:00:27:13:b6:38 (Oracle VirtualBox virtual NIC)
Too many fingerprints match this host to give specific OS details
Network Distance: 1 hop

TRACEROUTE
HOP RTT ADDRESS
1 3.62 ms 192.168.1.2

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/
Nmap done: 1 IP address (1 host up) scanned in 32.26 seconds
```

Figure 4.28: Nmap scan result in IDS mode

Nmap completed the scan but something changed compared to Sprint 3 , this time all 1000 ports are reported as **"ignored"** and port 80 no longer appears open.

Suricata detected the scan and the firewall began interfering with Nmap's ability to fingerprint the target. In Sprint 3, port 80 was clearly visible. Now it isn't.

```
(marion@kali)-[~]
$ hydra -l admin -P /usr/share/wordlists/rockyou.txt 192.168.1.2 http-get /
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organi
zations, or for illegal purposes (this is non-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-06-11 22:21:01
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (l:1/p:14344399), ~896525 tries per ta
sk
[DATA] attacking http-get://192.168.1.2:80/
[ERROR] all children were disabled due to too many connection errors
0 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-06-11 22:21:35
```

Figure 4.29: Hydra stopped by Suricata

In Sprint 3, Hydra ran freely and tested millions of passwords. This time it returned **"all children were disabled due to too many connection errors"** and found **0 valid passwords**.

Suricata detected the flood of connection attempts and the firewall started refusing Hydra's connections before it could complete a single attempt.

Services: Intrusion Detection: Administration									
Settings Download Rules User defined Alerts Schedule									
Search [icon] [icon] 2026/06/11 22:20 50 [icon]									
Timestamp	SID	Action	Interface	Source	Port	Destination	Port	Alert	Info
2026-06-11T22:21:05.477818+0...	429496...	allowed	WAN	192.168.1.156	35008	192.168.1.2	80	Nmap Port Scan Detected	[icon]
2026-06-11T22:21:05.477818+0...	429496...	allowed	WAN	192.168.1.156	35008	192.168.1.2	80	Hydra Brute-Force Detected por...	[icon]
2026-06-11T22:21:05.477066+0...	429496...	allowed	WAN	192.168.1.156	35000	192.168.1.2	80	Nmap Port Scan Detected	[icon]
2026-06-11T22:21:05.477066+0...	429496...	allowed	WAN	192.168.1.156	35000	192.168.1.2	80	Hydra Brute-Force Detected por...	[icon]
2026-06-11T22:21:05.476689+0...	429496...	allowed	WAN	192.168.1.156	34984	192.168.1.2	80	Nmap Port Scan Detected	[icon]
2026-06-11T22:21:05.476689+0...	429496...	allowed	WAN	192.168.1.156	34984	192.168.1.2	80	Hydra Brute-Force Detected por...	[icon]
2026-06-11T22:21:05.475590+0...	429496...	allowed	WAN	192.168.1.156	34972	192.168.1.2	80	Nmap Port Scan Detected	[icon]

Figure 4.30: Suricata Alerts tab: both rules firing

Both custom rules are triggering in real time : **"Nmap Port Scan Detected"** and **"Hydra Brute-Force Detected"** alerts are appearing on the WAN interface, with Kali (192.168.1.156) as the source targeting 192.168.1.2 on port 80.

These three results confirm the custom rules are working as expected. Suricata is detecting both attacks automatically and already impacting the attacker's ability to operate : **the IDS is ready to be switched to full blocking mode.**

2.3 Transition to IPS Mode

Until now Suricata was running in IDS mode : it detects and logs attacks but lets them through. Switching to IPS mode changes that completely.

Instead of just watching, Suricata will now actively drop malicious packets before they ever reach the DMZ server.

2.3.1 Switch Suricata to Inline IPS Mode

Capture mode switched to Netmap (IPS) Right now Suricata sees the attack, logs it, but does nothing to stop it. **Switching to Inline IPS mode** puts Suricata directly in the traffic path , every packet must pass through Suricata before reaching its destination, **giving it the power to drop packets in real-time.**

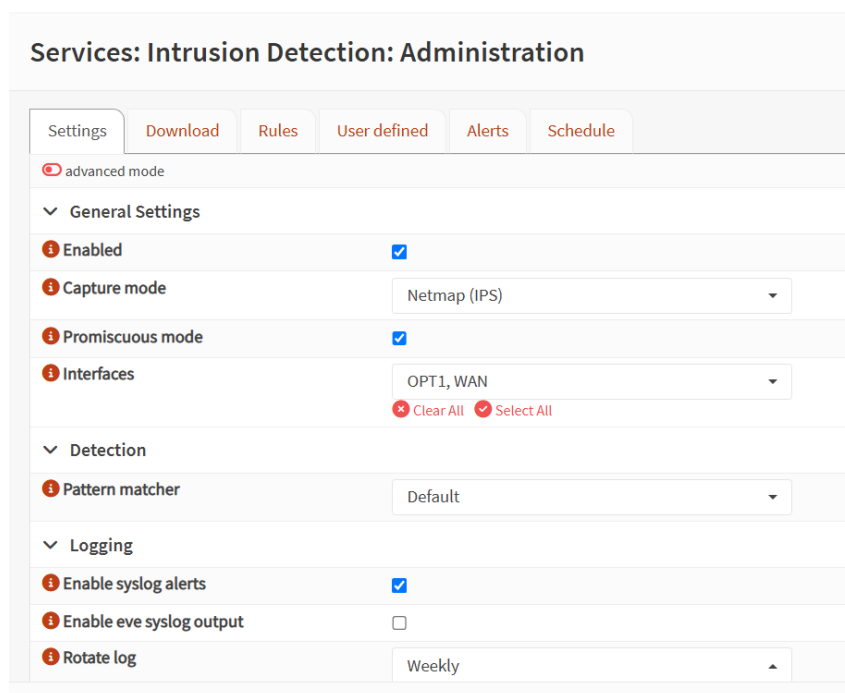


Figure 4.31: Suricata switched to Netmap (IPS) mode

We chose Netmap (IPS) because it operates directly at the network driver level without requiring extra configuration, and it is the most compatible mode for virtualized environments like VirtualBox.

- **Hardware offloading disabled in Interfaces Settings**

Netmap requires hardware offloading to be disabled to function correctly. Because without disabling it , the network card processes packets before Suricata even sees them

, so Suricata receives incomplete or modified packets and cannot inspect or drop them properly.

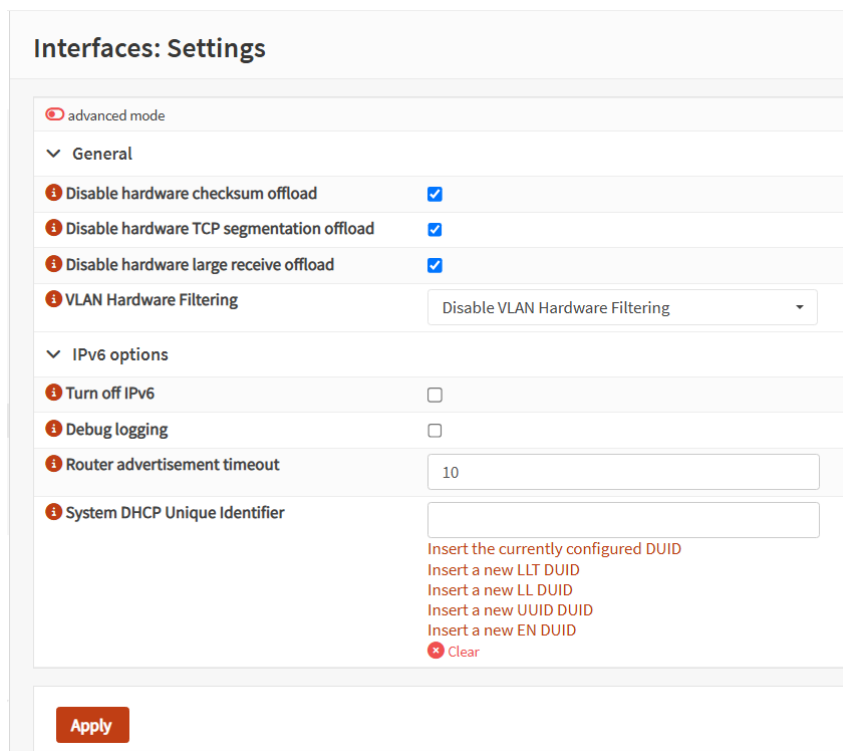


Figure 4.32: Hardware offloading disabled

All three hardware offload options are disabled, which is a required step for Netmap IPS mode in a virtualized environment.

2.3.2 Change Rules from Alert to Drop

Switching to IPS mode alone is not enough , we also need to **tell Suricata what to do when it matches an attack**. Our custom rules are still set to Alert, meaning Suricata logs the attack but still lets the packets through.

Changing to Drop means Suricata silently kills the connection the moment a rule matches , the attacker gets no response, no data, nothing.

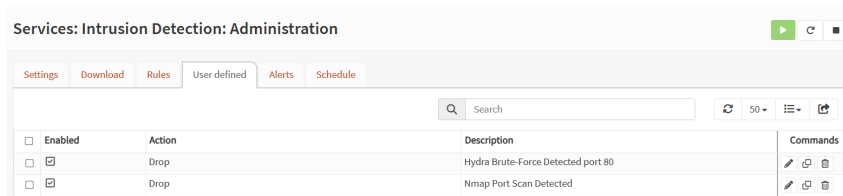


Figure 4.33: Custom rules set to Drop

Both custom rules : Nmap Port Scan Detected and Hydra Brute-Force Detected , are now set to **Drop**.

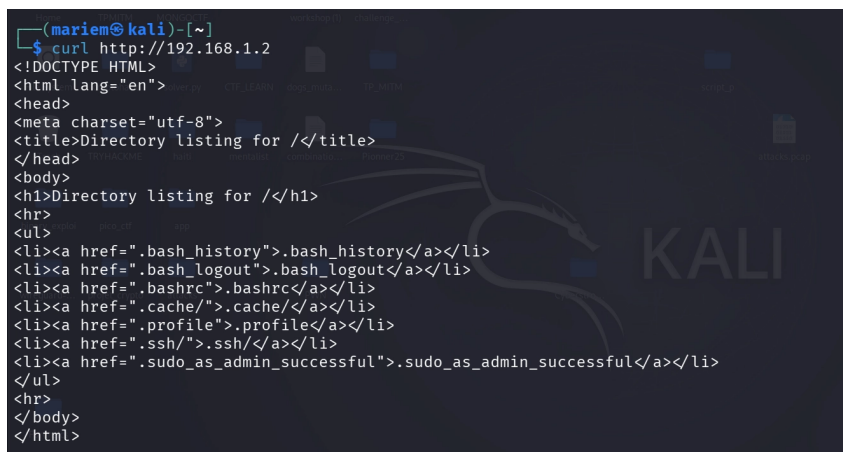
From this point forward, any attack traffic is dropped at the firewall before it reaches the server.

2.3.3 Verify No False Positives

Before declaring the IPS fully operational, we need to confirm that only real attack traffic gets dropped and that a legitimate user browsing the web server can still access it normally.

A false positive here would mean Suricata is blocking real users, which is just as harmful as missing an attack.

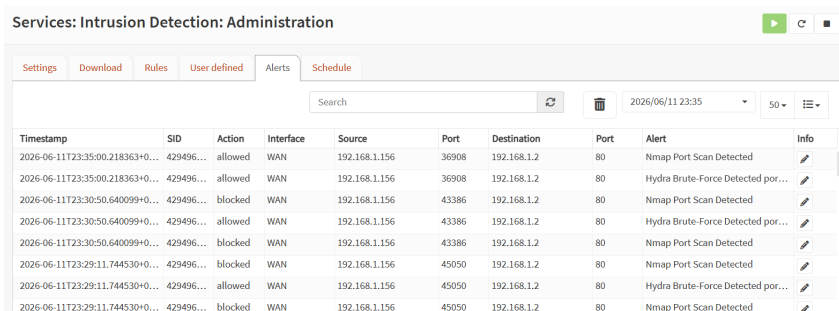
The Ubuntu HTTP server must be running first, since the NAT rule forwards port 80 traffic from OPNsense directly to the Ubuntu server at 10.0.2.10:80. If nothing is listening on port 80, the connection fails regardless of Suricata.



```
(marion@kali)-[~]
$ curl http://192.168.1.2
<!DOCTYPE HTML>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Directory listing for /</title>
</head>
<body>
<h1>Directory listing for /</h1>
<hr>
<ul>
<li><a href=".bash_history">.bash_history</a></li>
<li><a href=".bash_logout">.bash_logout</a></li>
<li><a href=".bashrc">.bashrc</a></li>
<li><a href=".cache/">.cache/</a></li>
<li><a href=".profile">.profile</a></li>
<li><a href=".ssh/">.ssh/</a></li>
<li><a href=".sudo_as_admin_successful">.sudo_as_admin_successful</a></li>
</ul>
<hr>
</body>
</html>
```

Figure 4.34: curl from Kali returns the web server page

The curl request to **http://192.168.1.2** successfully returned the Ubuntu web server directory listing, legitimate HTTP traffic passed through the IPS completely unblocked.



Services: Intrusion Detection: Administration

Settings Download Rules User defined Alerts Schedule

Search 2026/06/11 23:35 50

Timestamp	SID	Action	Interface	Source	Port	Destination	Port	Alert	Info
2026-06-11T23:35:00.218363+0...	429496...	allowed	WAN	192.168.1.156	36908	192.168.1.2	80	Nmap Port Scan Detected	
2026-06-11T23:35:00.218363+0...	429496...	allowed	WAN	192.168.1.156	36908	192.168.1.2	80	Hydra Brute-Force Detected por...	
2026-06-11T23:30:50.640099+0...	429496...	blocked	WAN	192.168.1.156	43386	192.168.1.2	80	Nmap Port Scan Detected	
2026-06-11T23:30:50.640099+0...	429496...	allowed	WAN	192.168.1.156	43386	192.168.1.2	80	Hydra Brute-Force Detected por...	
2026-06-11T23:30:50.640099+0...	429496...	blocked	WAN	192.168.1.156	43386	192.168.1.2	80	Nmap Port Scan Detected	
2026-06-11T23:29:11.744530+0...	429496...	blocked	WAN	192.168.1.156	45050	192.168.1.2	80	Nmap Port Scan Detected	
2026-06-11T23:29:11.744530+0...	429496...	allowed	WAN	192.168.1.156	45050	192.168.1.2	80	Hydra Brute-Force Detected por...	
2026-06-11T23:29:11.744530+0...	429496...	blocked	WAN	192.168.1.156	45050	192.168.1.2	80	Nmap Port Scan Detected	

Figure 4.35: Suricata Alerts tab: no false positives

The alerts log shows only Nmap and Hydra events from previous tests. The legitimate curl request does not appear anywhere : confirming that **normal HTTP traffic is completely invisible to our drop rules**.

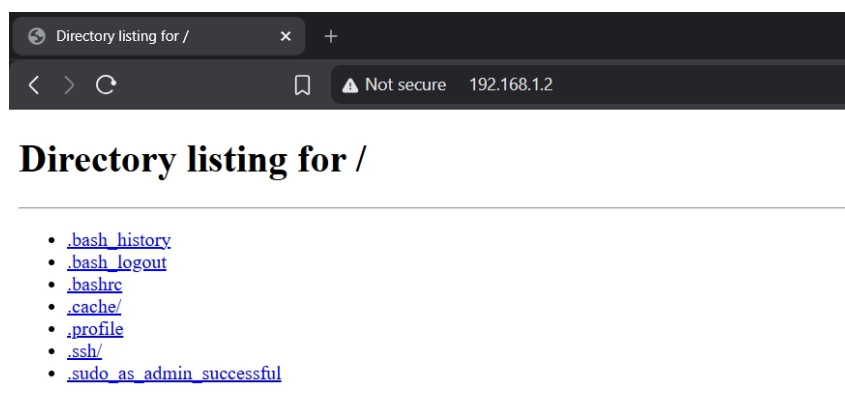


Figure 4.36: Web server accessible from Windows browser

Accessing **192.168.1.2** from a Windows browser successfully loads the Ubuntu DMZ directory listing : confirming legitimate traffic passes through the IPS without any interference.

2.4 Final Test: All Three Attacks Blocked

This is the final validation of Sprint 4. We re-launch all three attacks from Kali , the same attacks from Sprint 3 , but this time Suricata is running in IPS mode with Drop rules active. The firewall must now **block every attack automatically before it reaches the DMZ server**, while keeping the web server accessible to legitimate traffic.

2.4.1 Nmap Scan

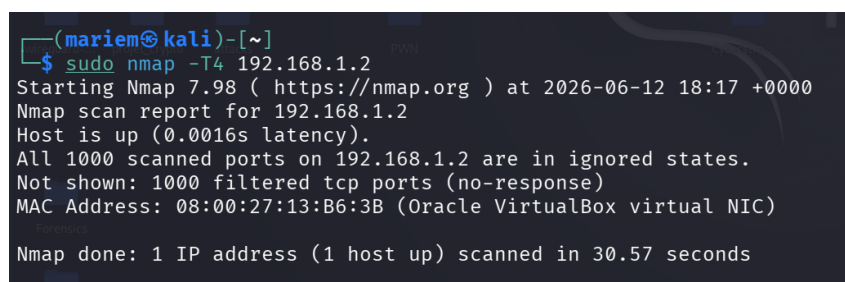


Figure 4.37: Nmap scan: all ports filtered

In Sprint 3, Nmap found port 80 open and identified the Python web server. This time all 1000 ports come back as filtered with no response .

Suricata intercepted the scan packets before they could reach the target. The attacker gets zero information about the network.

2.4.2 hping3 SYN Flood

```
(mariem@kali)-[~]
$ sudo hping3 -S --flood -p 80 192.168.1.2
HPING 192.168.1.2 (eth0 192.168.1.2): S set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown
^C
— 192.168.1.2 hping statistic —
1905887 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
```

Figure 4.38: hping3: 100% packet loss, 0 replies received

The SYN flood is blocked by the **ET open/emerging-dos** ruleset downloaded earlier, which detects an abnormal volume of SYN packets, no custom rule was needed for this attack.

hping3 transmitted nearly 2 million SYN packets and received **0 replies**, **100% packet loss**. Every single flood packet was silently dropped by Suricata before reaching the Ubuntu DMZ server. In Sprint 3 the SYN flood overwhelmed the server. Now it doesn't touch it.

2.4.3 Hydra Brute-Force

```
(mariem@kali)-[~]
$ hydra -l admin -P /usr/share/wordlists/rockyou.txt 192.168.1.2 http-get /
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for
illegal purposes (this is non-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-06-12 18:46:39
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (l:1/p:14344399), ~896525 tries per task
[DATA] attacking http-get://192.168.1.2:80/
[ERROR] all children were disabled due to too many connection errors
0 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-06-12 18:47:14
```

Figure 4.39: Hydra: 0 valid passwords

In Sprint 3, Hydra successfully found 16 valid passwords in 2 seconds. This time it returned **"all children disabled due to too many connection errors"** and **0 valid passwords found**. Suricata detected the rapid HTTP authentication attempts and dropped every connection : Hydra couldn't complete a single login attempt.

2.4.4 Suricata Alerts Confirm Full Blocking

Services: Intrusion Detection: Administration

Settings Download Rules User defined Alerts Schedule

Search 2026/06/12 18:46 50

Timestamp	SID	Action	Interface	Source	Port	Destination	Port	Alert	Info
2026-06-12T18:46:41.363516+0...	429496...	blocked	WAN	192.168.1.156	42626	192.168.1.2	80	Hydra Brute-Force Detected por...	
2026-06-12T18:46:41.363516+0...	429496...	blocked	WAN	192.168.1.156	42626	192.168.1.2	80	Nmap Port Scan Detected	
2026-06-12T18:46:41.363516+0...	429496...	blocked	WAN	192.168.1.156	42626	192.168.1.2	80	Hydra Brute-Force Detected por...	
2026-06-12T18:46:41.363516+0...	429496...	blocked	WAN	192.168.1.156	42624	192.168.1.2	80	Hydra Brute-Force Detected por...	
2026-06-12T18:46:41.363516+0...	429496...	blocked	WAN	192.168.1.156	42624	192.168.1.2	80	Nmap Port Scan Detected	
2026-06-12T18:46:41.363516+0...	429496...	blocked	WAN	192.168.1.156	42624	192.168.1.2	80	Hydra Brute-Force Detected por...	
2026-06-12T18:46:41.363516+0...	429496...	blocked	WAN	192.168.1.156	42618	192.168.1.2	80	Hydra Brute-Force Detected por...	
2026-06-12T18:46:41.363516+0...	429496...	blocked	WAN	192.168.1.156	42618	192.168.1.2	80	Nmap Port Scan Detected	
2026-06-12T18:46:41.363516+0...	429496...	blocked	WAN	192.168.1.156	42618	192.168.1.2	80	Hydra Brute-Force Detected por...	
2026-06-12T18:46:41.363516+0...	429496...	blocked	WAN	192.168.1.156	42608	192.168.1.2	80	Hydra Brute-Force Detected por...	
2026-06-12T18:46:41.363516+0...	429496...	blocked	WAN	192.168.1.156	42608	192.168.1.2	80	Nmap Port Scan Detected	

Showing 1 to 50 of 51 entries

Figure 4.40: Suricata Alerts

Every single entry in the Alerts log shows "blocked" , both Nmap Port Scan Detected and Hydra Brute-Force Detected across 51 entries. Kali (192.168.1.156) kept sending attacks, but every packet was killed at the firewall before reaching the DMZ server.

The lab has successfully transformed **from a passive observer into an active defense system**.

2.5 Bonus: Syslog Telemetry Forwarding

Until now, Suricata's alerts were **only visible locally** on the firewall , which is not realistic in a real security environment.

This bonus solves that by configuring Suricata to forward every alert over the network to a central Windows PC using **Syslog** (a standard protocol used to send log messages from one machine to another over the network), where a Python script catches and displays them in real-time. This simulates a **basic SIEM ingestion pipeline** : the full chain from detection on the firewall, through the network, all the way to a centralized collector , exactly how enterprise security monitoring works.

2.5.1 Enable Suricata Syslog Output

Suricata is already generating alerts, but they stay on the firewall. This step enables **Suricata's Syslog output** so that its alerts are passed to the OPNsense system log in real-time, ready to be forwarded in the next step.

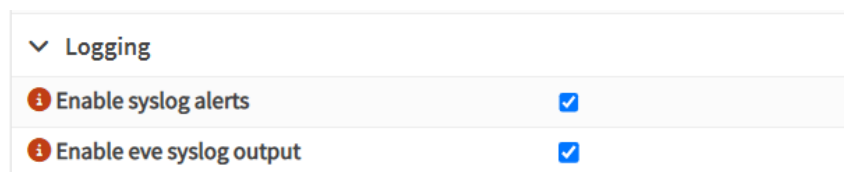


Figure 4.41: Suricata Syslog output enabled

Both options are now checked. "**Enable syslog alerts**" sends a basic alert message to the system log every time a rule triggers. "**Enable eve syslog output**" additionally forwards detailed JSON-formatted alerts via Syslog including source IP, destination IP, port, protocol, and the rule that fired.

Together, these two settings tell Suricata to **stop keeping its alerts to itself and start broadcasting them to the system**.

2.5.2 Configure OPNsense Syslog Forwarder

Suricata is now writing alerts to the OPNsense system log, but they are still stuck on the firewall. This step tells OPNsense to forward those logs over the network to the Windows PC (**192.168.2.10**) via **UDP** on **port 514** : the standard Syslog port defined since 1983. UDP is used because it is lightweight and fast: for real-time alert forwarding, speed matters more than guaranteed delivery.

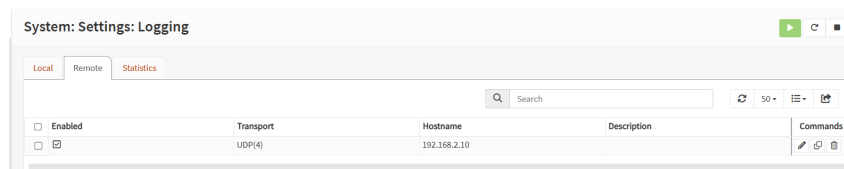


Figure 4.42: OPNsense remote Syslog forwarder configured

OPNsense is now configured to forward all system logs including Suricata alerts to the Windows PC via **UDP(4)** on **port 514**. From this moment, every alert Suricata generates will travel from the firewall through the LAN and arrive at the Windows machine, where the Python listener is waiting to catch it.

2.5.3 Write a Python UDP Listener on the Windows Host

OPNsense is forwarding alerts over the network, but nothing on the Windows side is listening yet , the alerts arrive and get ignored.

This step fixes that with a small Python script that opens UDP port 514 and waits for incoming messages. The moment OPNsense sends a Suricata alert, the script catches it and prints it on screen in real-time.

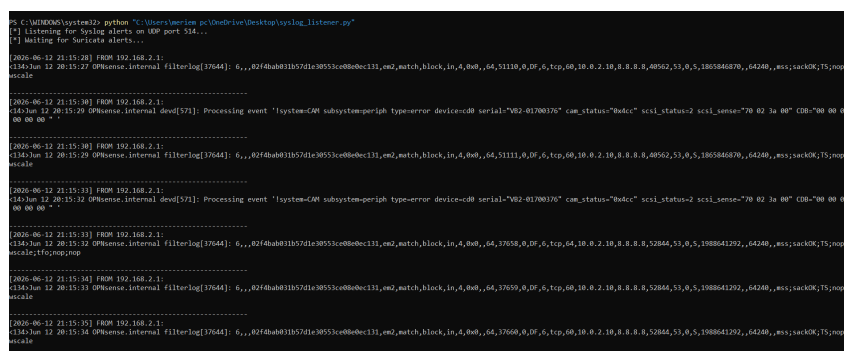


Figure 4.43: Python listener running and receiving logs

The script is running and logs are already arriving from OPNsense (192.168.2.1). These are general system logs , not Suricata alerts yet, since no attack has been launched. What matters is that the channel is open and working.

2.5.4 Confirm Alerts Appear in Real-Time

This is the final validation of the bonus. With Suricata generating alerts, OPNsense forwarding them via **Syslog**, and the Python listener running on Windows, we now launch a real attack to prove the full pipeline works end to end without touching the firewall at all.

- Identifying the correct target IP

Initial attempts used 10.0.2.10 (Ubuntu DMZ) directly from Kali, which failed because Kali cannot reach private DMZ addresses. The correct target is the OPNsense WAN IP (**192.168.1.2**).

- Adding the syslog-ng configuration file

By default, **syslog-ng** (the log forwarding service running behind the scenes on OPNsense) only forwards general system logs. This configuration file tells it to also read Suricata's **eve.json** (the file where Suricata writes all its alerts in real-time) and forward every new entry to the Windows Python listener.

```
root@OPNsense:~ # cat /usr/local/etc/syslog-ng.conf.d/suricata-forward.conf
source s_suricata {
    file("/var/log/suricata/eve.json" follow-freq(1) flags(no-parse));
};
log {
    source(s_suricata);
    destination(d_fe161b17ba52436f8e4ccedba407fd3f);
};
root@OPNsense:~ #
```

Figure 4.44: syslog-ng Suricata forward config

The file reads Suricata's eve.json in real-time and forwards every new alert to **192.168.2.10 on UDP port 514**. Once syslog-ng was restarted, Suricata alerts started appearing in the Python listener automatically.

Launching the attack from Kali

```
(marion@kali)~$ curl -A "Nmap Scripting Engine" http://192.168.1.2
curl: (28) Failed to connect to 192.168.1.2 port 80 after 135140 ms: Could not connect to server
```

Figure 4.45: curl with malicious User-Agent sent from Kali

A curl request was sent from Kali (192.168.1.156) to the OPNsense WAN IP (192.168.1.2) with the User-Agent "Nmap Scripting Engine".

The connection failed , expected, since Suricata dropped the packet in IPS mode. But Suricata still detected and logged it, forwarding the alert through the pipeline.

Alert received on the Windows listener

```
[2020-06-13 17:18:54] FROM 192.168.2.1:
[17]>Jun 13 16:18:51 OPNsense.internal suricata[16734]: [Drop] [1:4294067293:1] Hydra Brute-Force Detected port 80 [Classification: null] [Priority: 3] (TCP 192.168.1.156:50464 -> 192.168.1.2:80)

[2020-06-13 17:18:54] FROM 192.168.2.1:
[17]>Jun 13 16:18:51 OPNsense.internal suricata[16734]: [Drop] [1:4294067294:1] Nmap Port Scan Detected [Classification: null] [Priority: 3] (TCP 192.168.1.156:50464 -> 192.168.1.2:80)
```

Figure 4.46: Suricata alerts appearing on Windows Python listener

Two alerts appeared instantly on the Windows listener : "Hydra Brute-Force Detected port 80" and "Nmap Port Scan Detected" , both showing **[Drop]** and the full traffic details: **192.168.1.156 → 192.168.1.2:80**.

The complete pipeline is confirmed: **Kali attacks → Suricata detects and blocks → alert forwarded via Syslog → Python listener displays in real-time**.

2.6 Conclusion

Sprint 4 marks the point where the lab stopped being a passive network and became **an active defense system**.

Starting from the attack knowledge built in **Sprint 3**, we deployed **Suricata** directly on the **OPNsense firewall**, configured it to monitor both the **WAN and DMZ** interfaces, and loaded it with community rulesets and custom rules targeting the exact attacks we had previously launched. Once validated in **IDS mode**, we switched to full **IPS mode** , giving Suricata the power to drop malicious packets before they ever reach the DMZ server.

The final test confirmed it: **Nmap** got no information, the **SYN flood** was absorbed without a single reply, and **Hydra** could not complete a single login attempt. Legitimate traffic continued to flow normally throughout.

As a bonus, we built a complete **Syslog forwarding pipeline** , from Suricata on the firewall to a **Python listener** on a Windows machine , simulating how real security teams centralize and monitor alerts across their infrastructure.

General Conclusion

This project was built from scratch : starting with nothing but a hypervisor and a few virtual machines, and ending with a **fully functional network security lab** that can detect, block, and log real attacks in real-time.

The first phase focused on building the foundation. We designed a segmented network with three distinct zones : **WAN, LAN, and DMZ** , separated by an **OPNsense firewall** with strict **access rules**. We deployed an **Ubuntu web server** in the DMZ, configured **NAT and port forwarding** to expose it to the outside, and established an **encrypted VPN tunnel** to allow secure remote administration from outside the network.

The second phase turned the lab into a **security system**. We first played the role of the attacker : using Kali Linux to launch three real attacks against the network: an **Nmap port scan** to map the infrastructure, an **hping3 SYN flood** to overwhelm the server, and a **Hydra brute-force** attack to crack the web server's credentials. We captured every attack in **Wireshark** and analyzed the exact packet signatures that each tool leaves behind.

We then **switched sides**. Using what we learned from Wireshark, we deployed **Suricata** on the firewall and wrote custom detection rules targeting the exact patterns we had just observed. Once we confirmed the rules were working, we switched Suricata to full **prevention mode** , and re-launched all three attacks. This time, every single packet was dropped before it reached the server.

Finally, we built a **Syslog forwarding pipeline** so that Suricata alerts are automatically sent to a central Windows machine and displayed in real-time by a **Python script** , simulating the kind of **centralized monitoring** used by real security teams.

This project covered the full cycle of network security: **build, attack, analyze, defend, and monitor**.